

Comunicação Serial



Comunicando

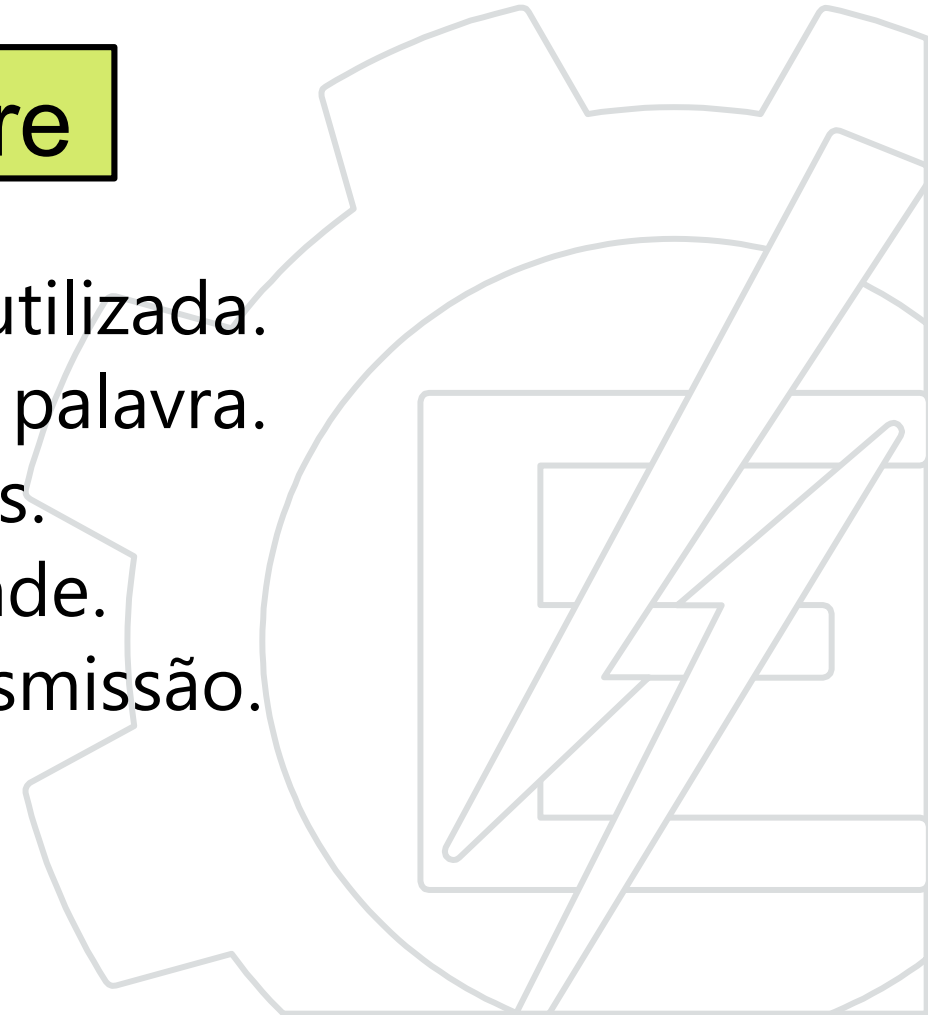
- Para que haja comunicação é necessário definir:

Hardware

- ✓ Tipo de conector.
- ✓ Pinos do conector.
- ✓ Níveis de tensão.
- ✓ ...

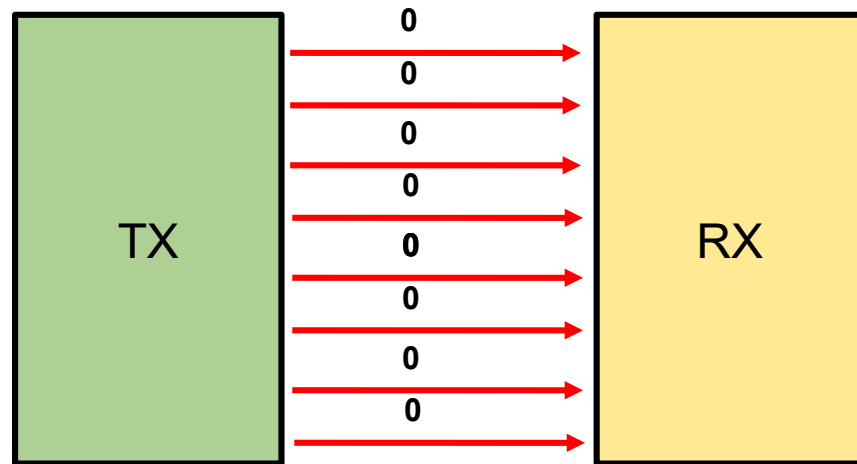
Software

- ✓ Codificação utilizada.
- ✓ Tamanho da palavra.
- ✓ Start/stop bits.
- ✓ Bit de paridade.
- ✓ Taxa de transmissão.
- ✓ ...

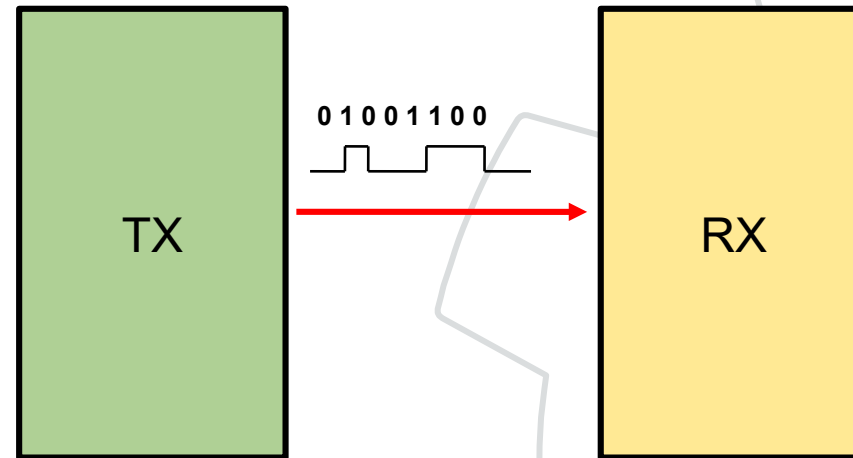


Transmissão paralela e serial

Tipos de Protocolos de comunicação

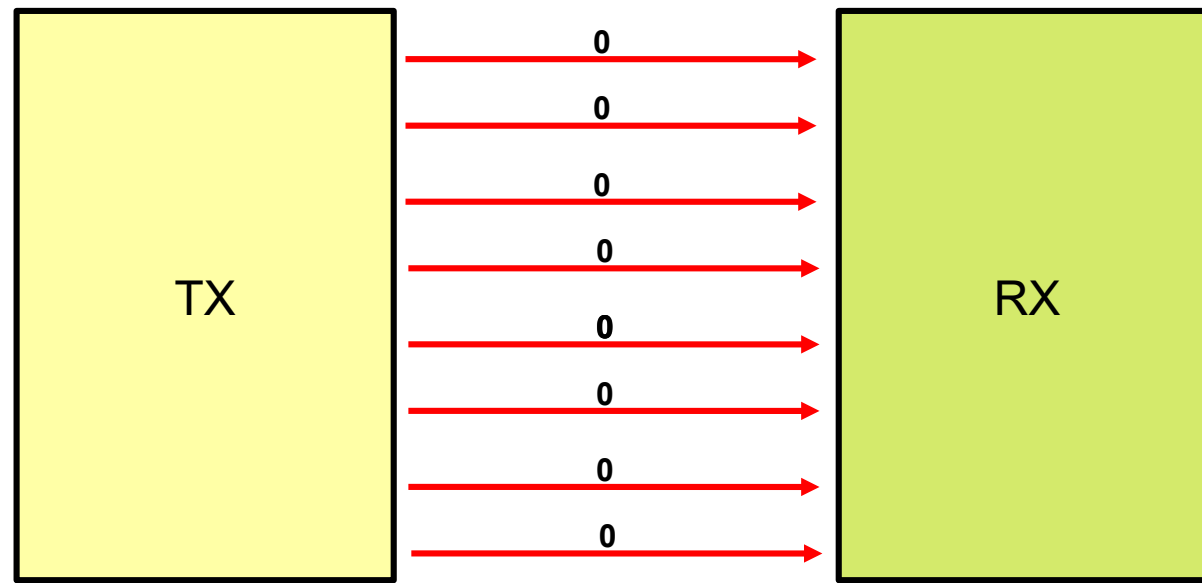


Comunicação paralela

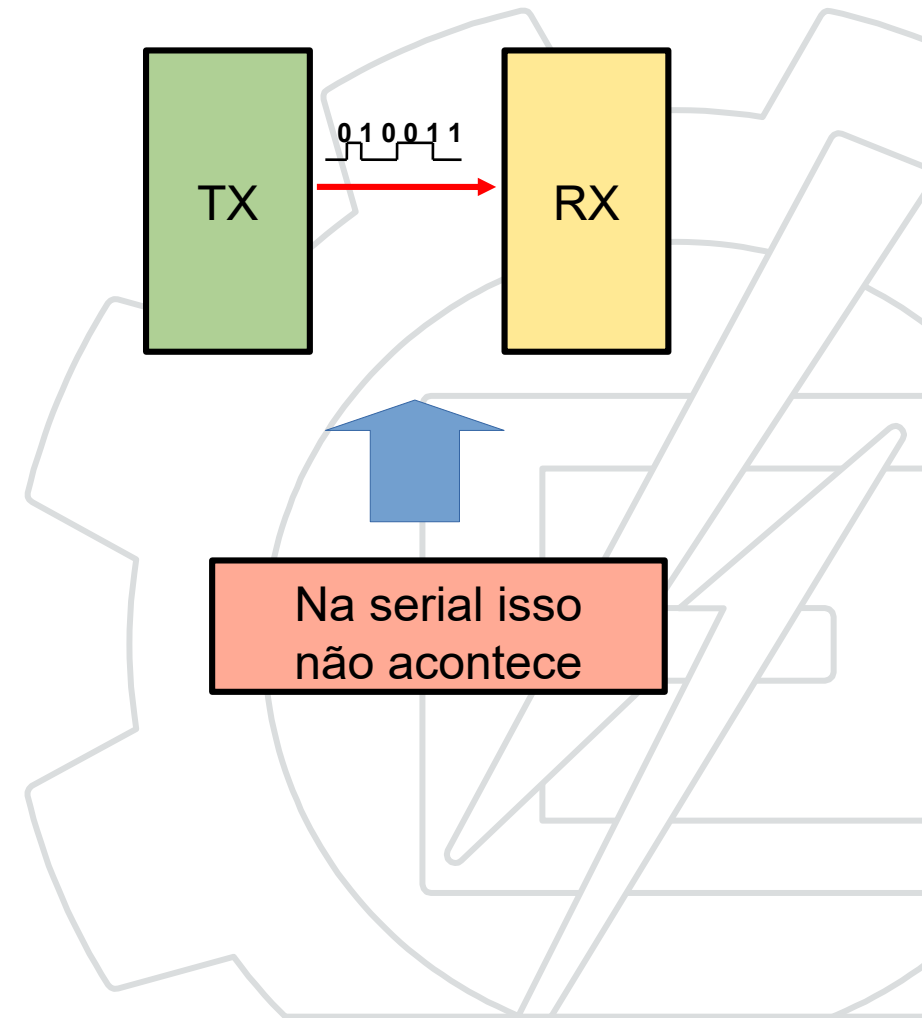


Comunicação serial

Uma vantagem da transmissão serial



Na Comunicação paralela se um sinal chegar antes do outro o sistema falha

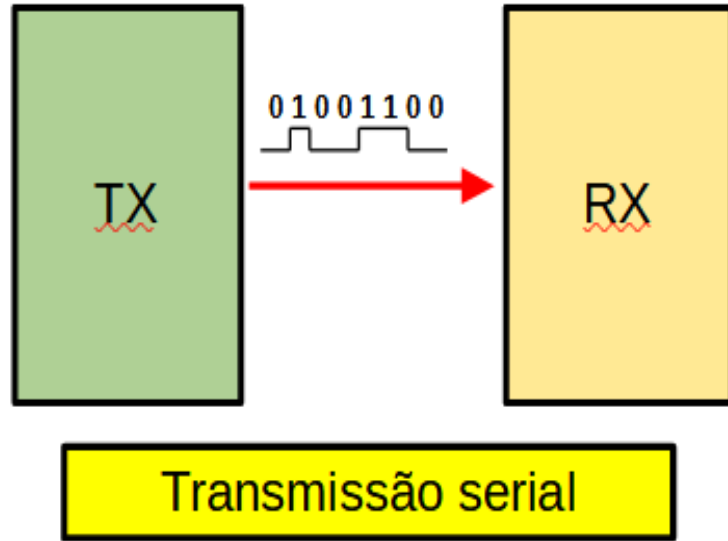


Na serial isso não acontece

Alguns Padrões de existentes

Protocolo	Taxa (bits/s)	Taxa (bytes/s)
Serial MIDI	31.25 kbit/s	3.9 kB/s
Serial EIA-232 max.	230.4 kbit/s	28.8 kB/s
Serial UART max	2.7648 Mbit/s	345.6 kB/s
I2C	3.4 Mbit/s	425 kB/s
Serial EIA-422 max.	10 Mbit/s	1.25 MB/s
SPI Bus (Up to 100MHz)	100 Mbit/s	12.5 MB/s
USB super speed (USB 3.0)	5 Gbit/s	625 MB/s
HDMI v. 1.3	10.2 Gbit/s	1.275 GB/s
Ultra DMA ATA 133 (paralelo 16 bits)	1,064 Mbit/s	133 MB/s
Serial ATA 3 (SATA-600)	6,000 Mbit/s	600 MB/s
Ultra-640 SCSI (paralelo 16 bits)	5,120 Mbit/s	640 MB/s
Serial Attached SCSI (SAS) 3	9,600 Mbit/s	1,200 MB/s

Comunicação básica serial

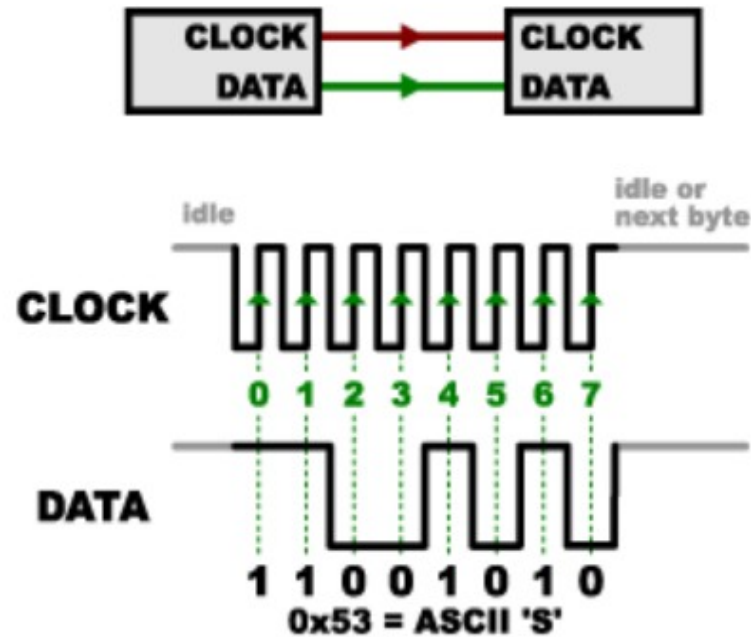


HDMI 1.4V
10,2Gb/s



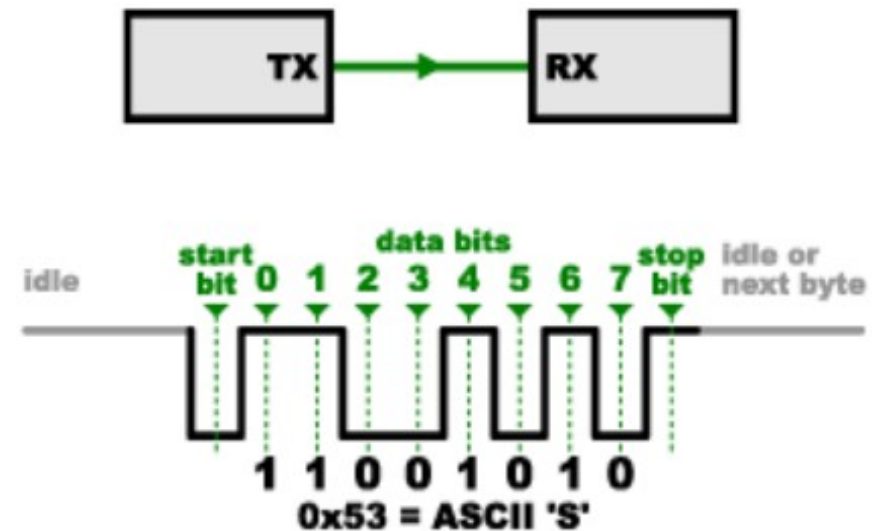
USB 3.0
5Gb/s

Transmissão serial



Fonte: <https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi>
Acesso em ago. 2016.

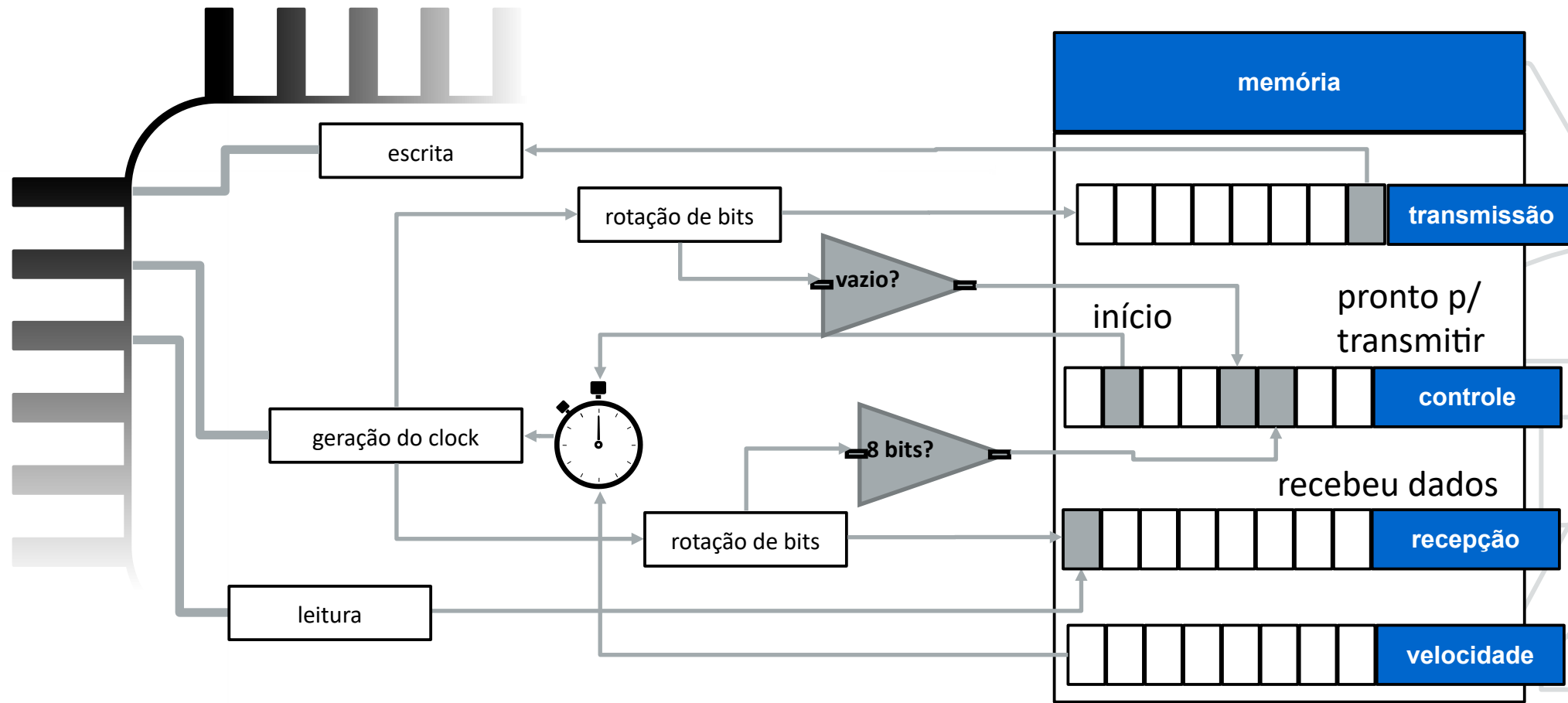
Serial síncrona



Fonte: <https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi>
Acesso em ago. 2016.

Serial assíncrona

O controle pode ser por Hardware



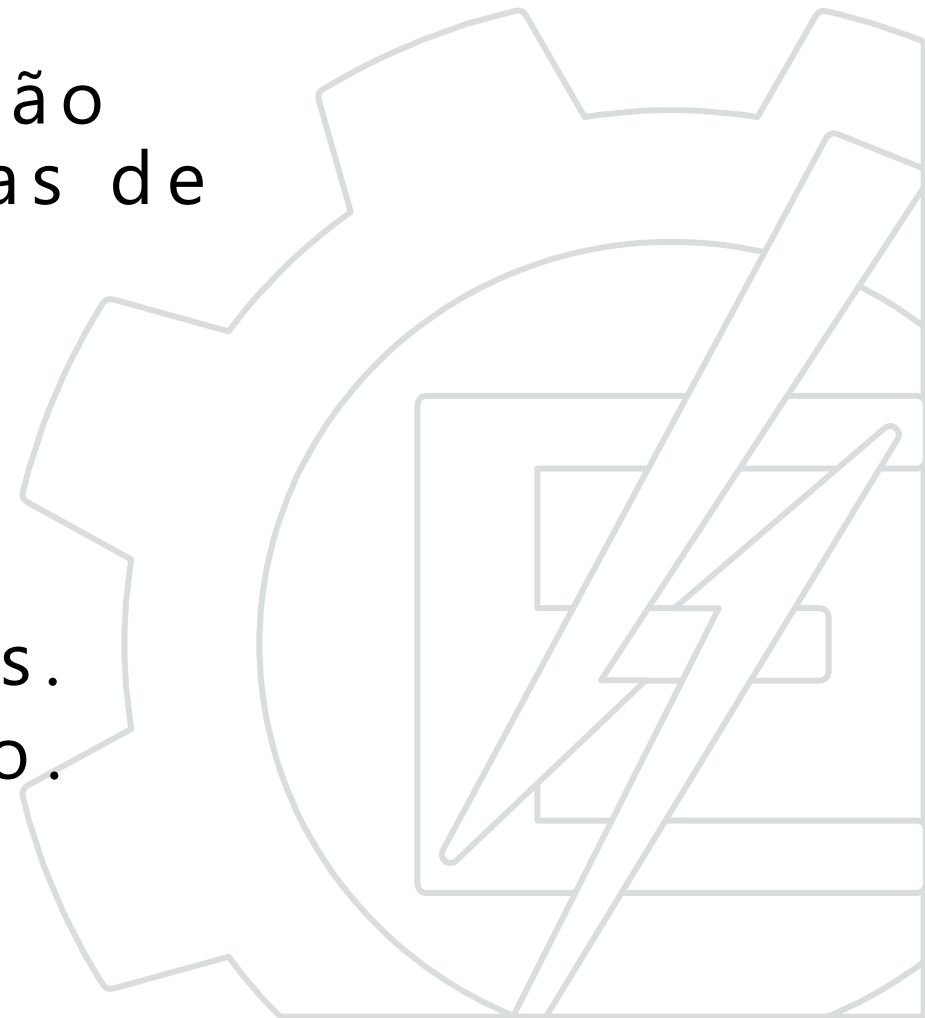
Comunicação Serial

- **Vantagens**

- Simplificação do hardware.
- Aumento da taxa de transmissão devido a redução de problemas de sincronia.
- Economia de cabos.

- **Desvantagens**

- Necessidade de circuitos extras.
- Maior complexidade no código.

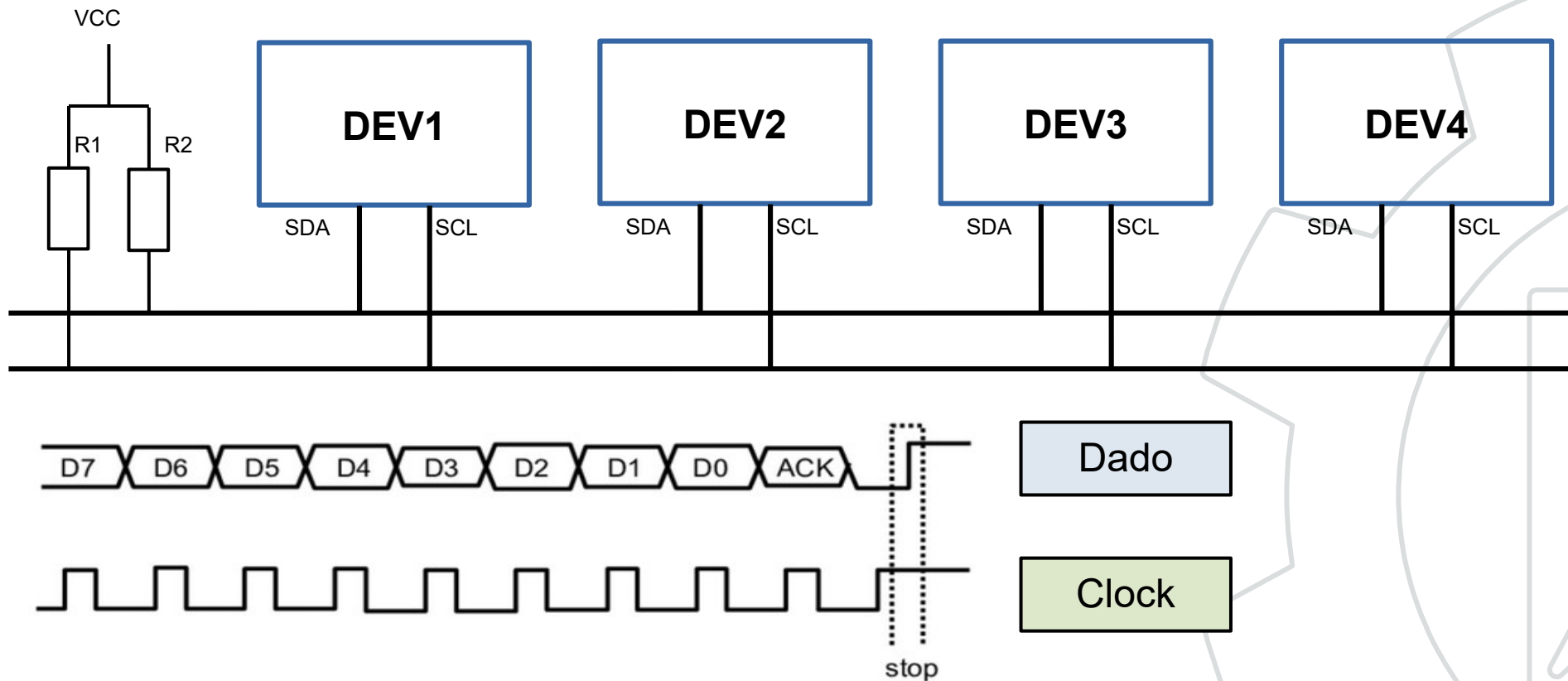


Protocollo I2C

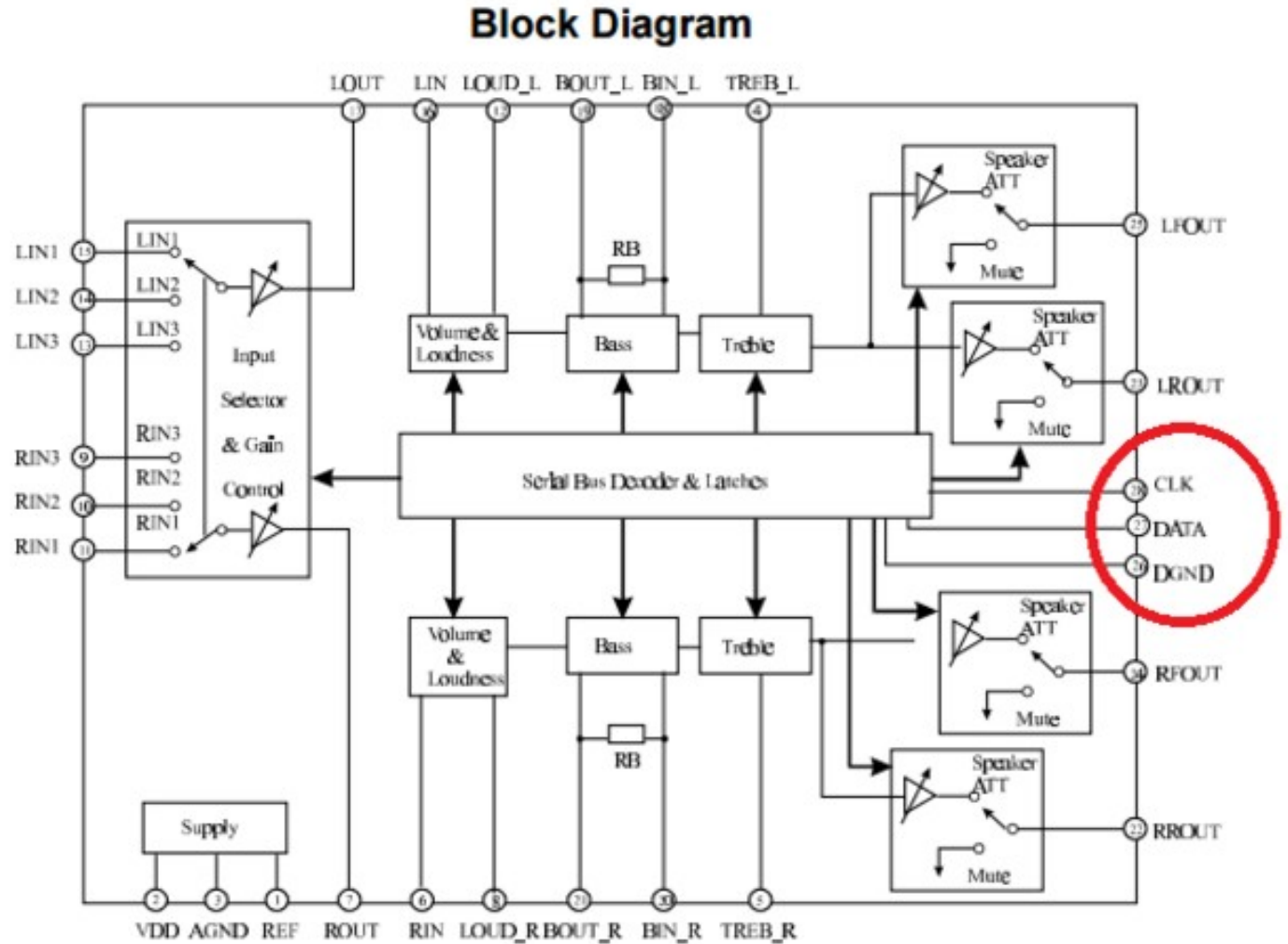
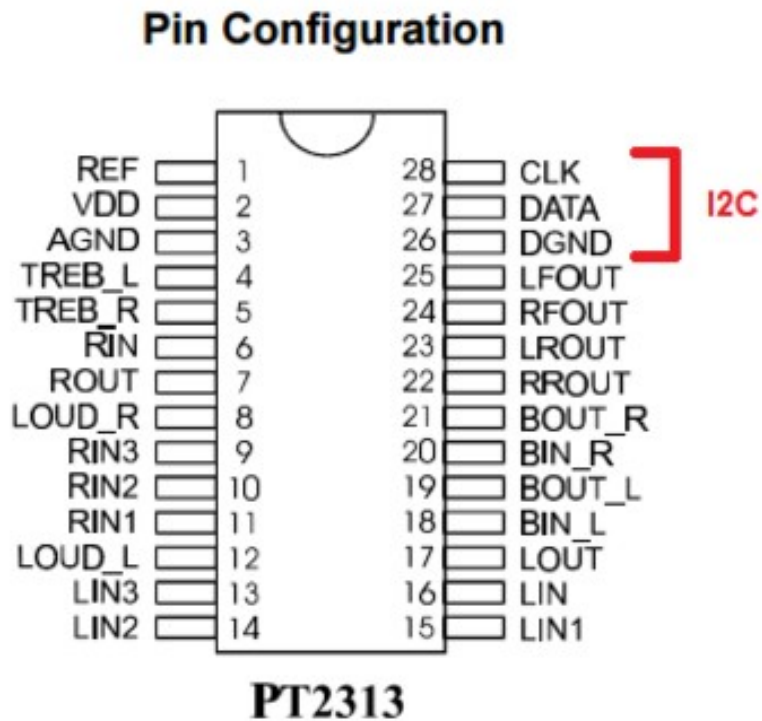


Comunicação serial I2C

- É um protocolo serial, síncrono e multi-mestre.



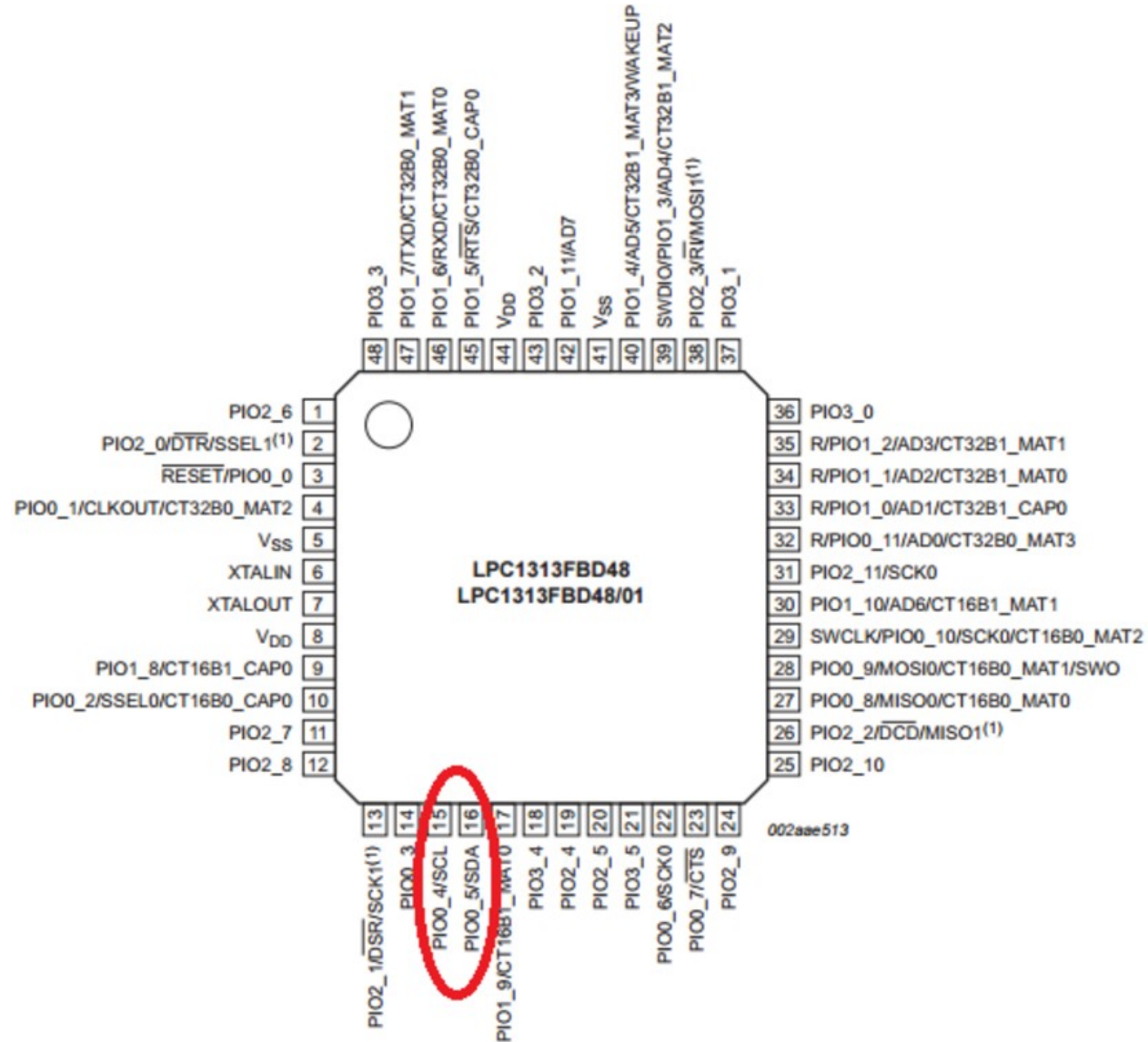
Comunicação serial I2C



Comunicação serial I2C



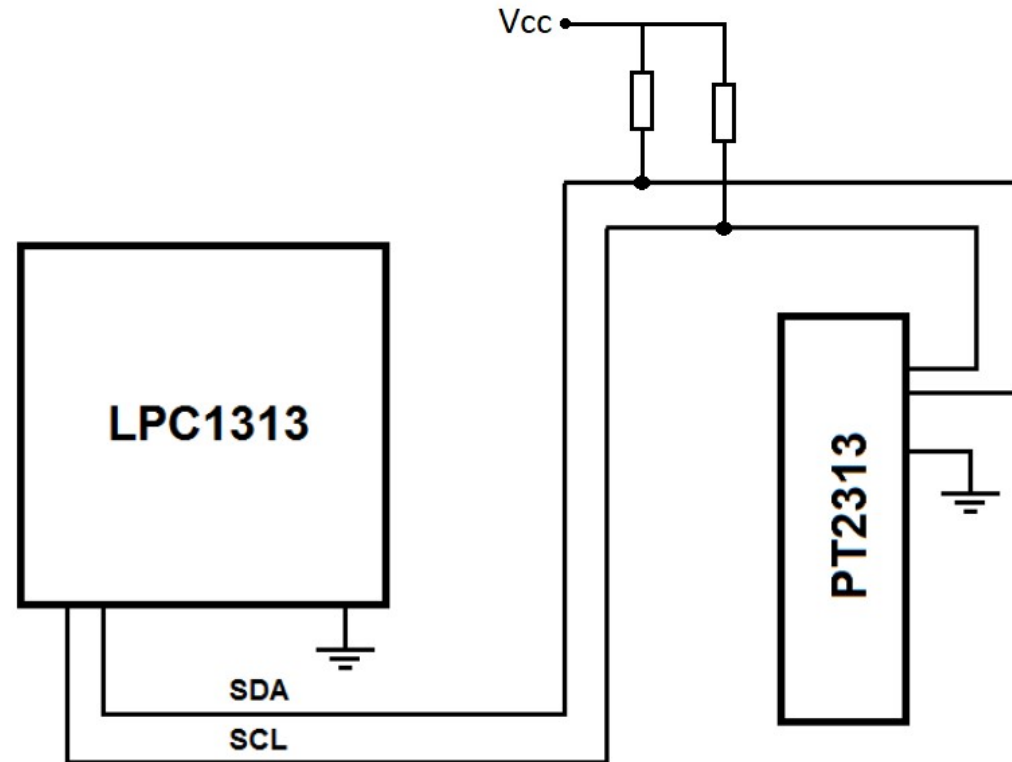
NXP LPC1313



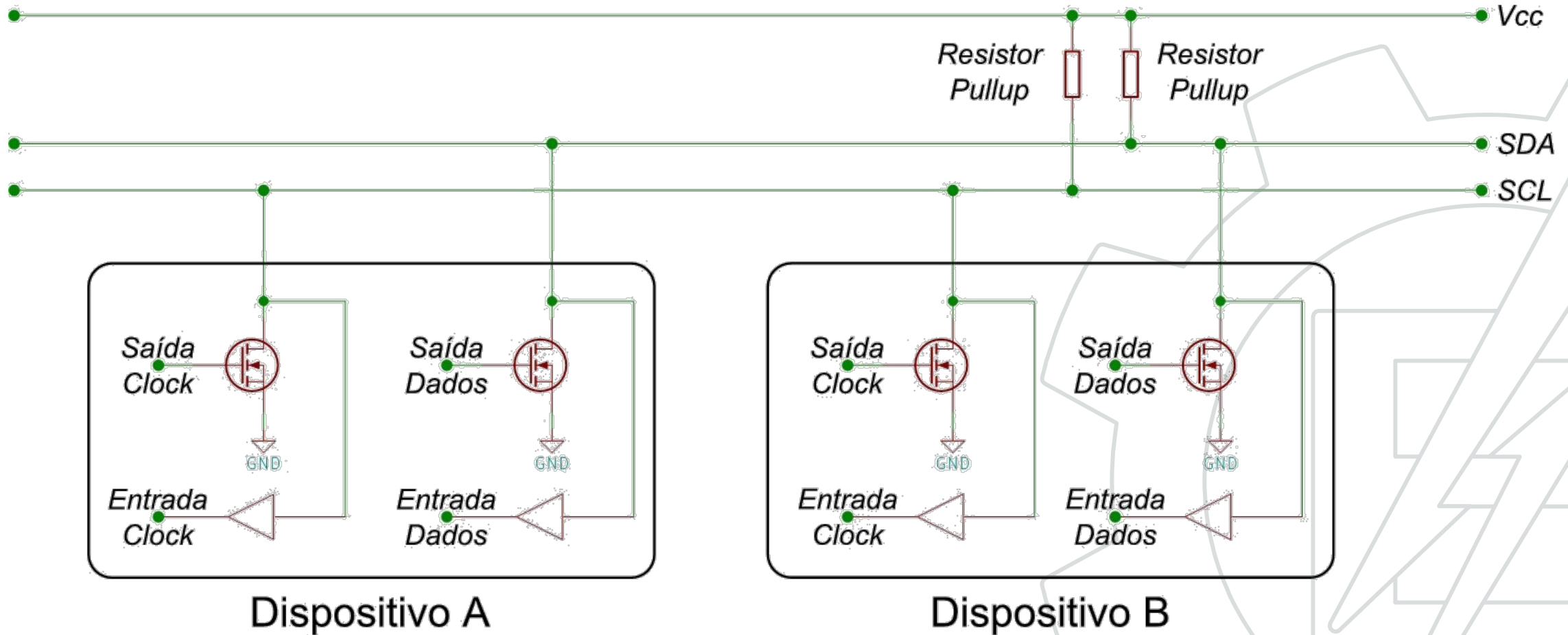
Comunicação serial I2C



NXP LPC1313

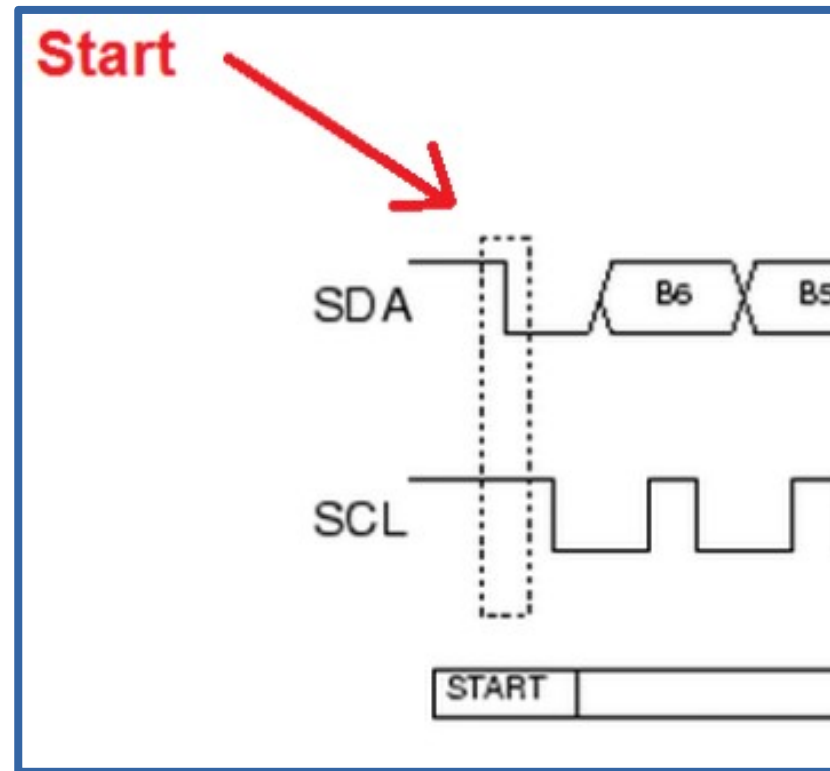


Comunicação serial I2C



Comunicação serial I2C

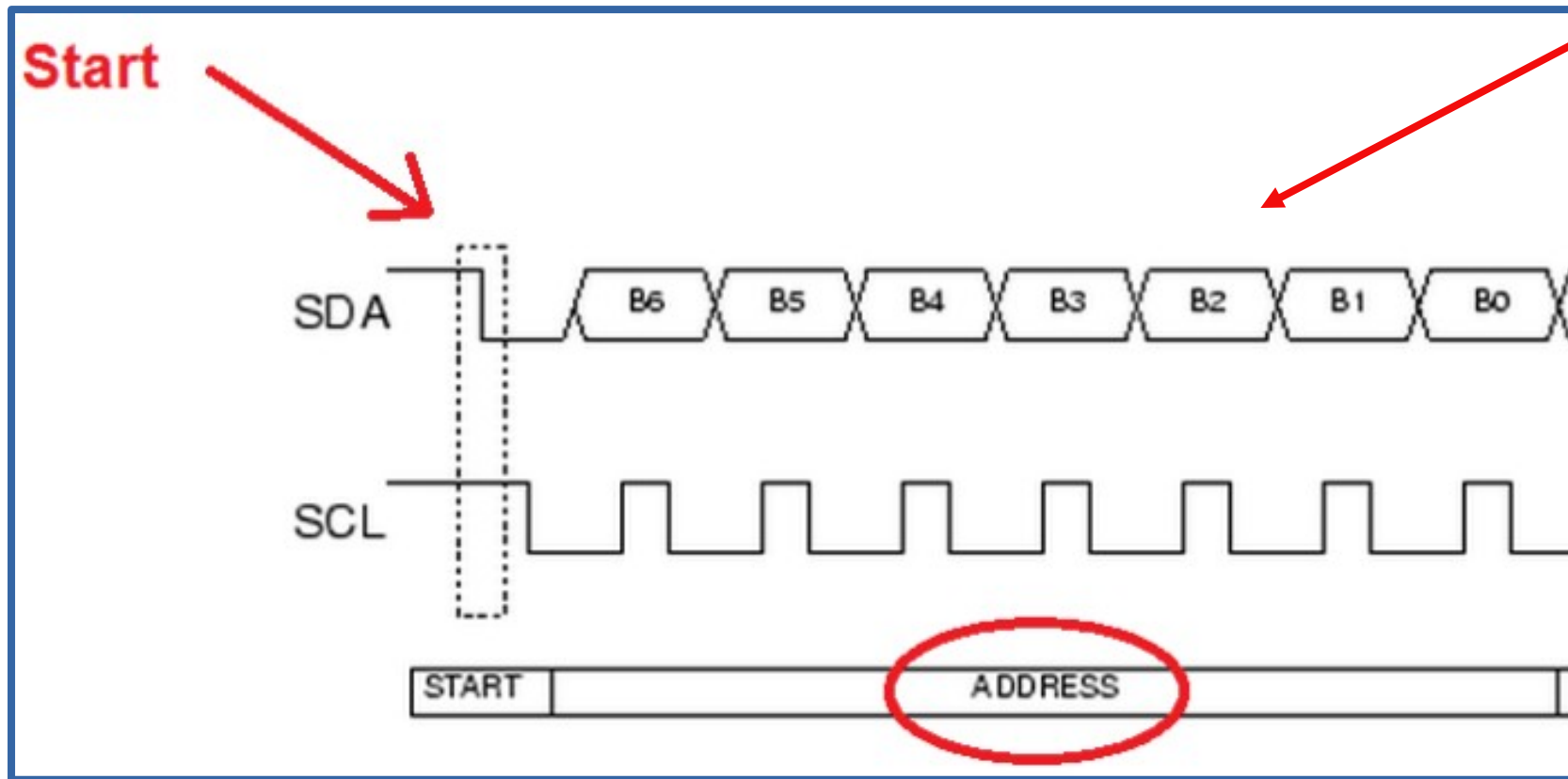
- A comunicação é iniciada quando SDA é colocado em nível baixo com SCL em nível alto.



Comunicação serial I2C

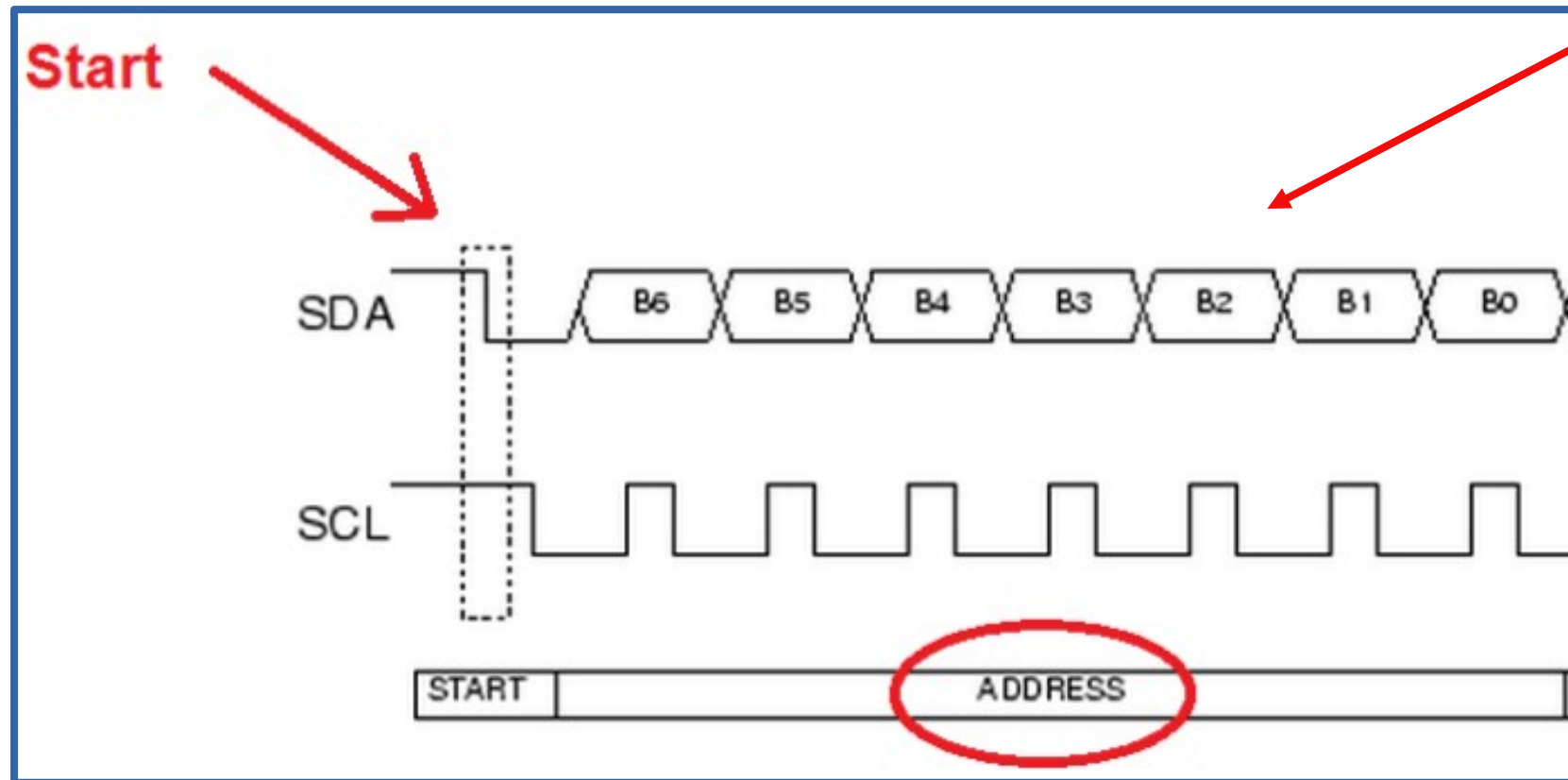
- Na sequência o endereço do dispositivo é enviado.

Endereço



Comunicação serial I2C

- Na sequência o endereço do dispositivo é enviado.



Endereço

Os dispositivos recebem essa informação e comparam com o endereço interno.

Comunicação serial I2C

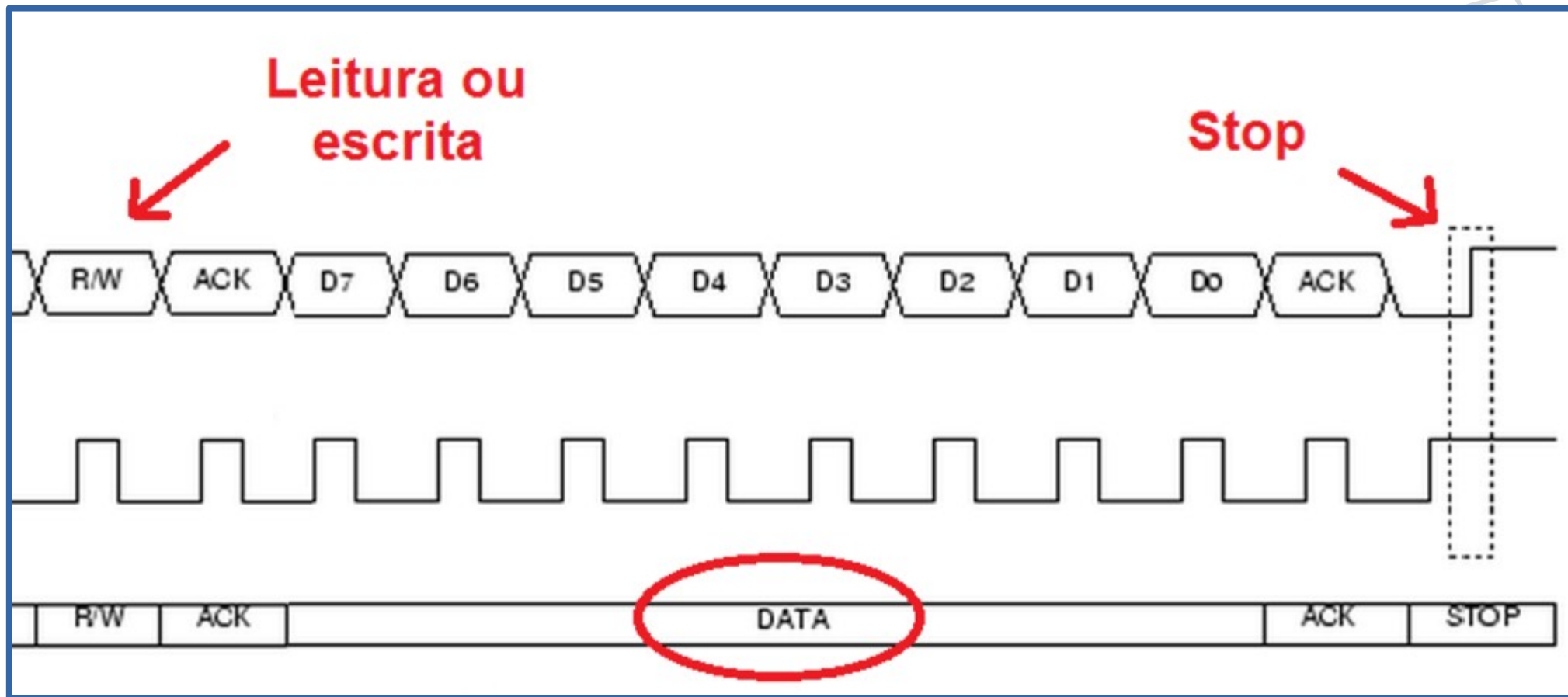
- Após o endereço um bit indica se é leitura ou escrita.



O dispositivo SLAVE responde com um ACK

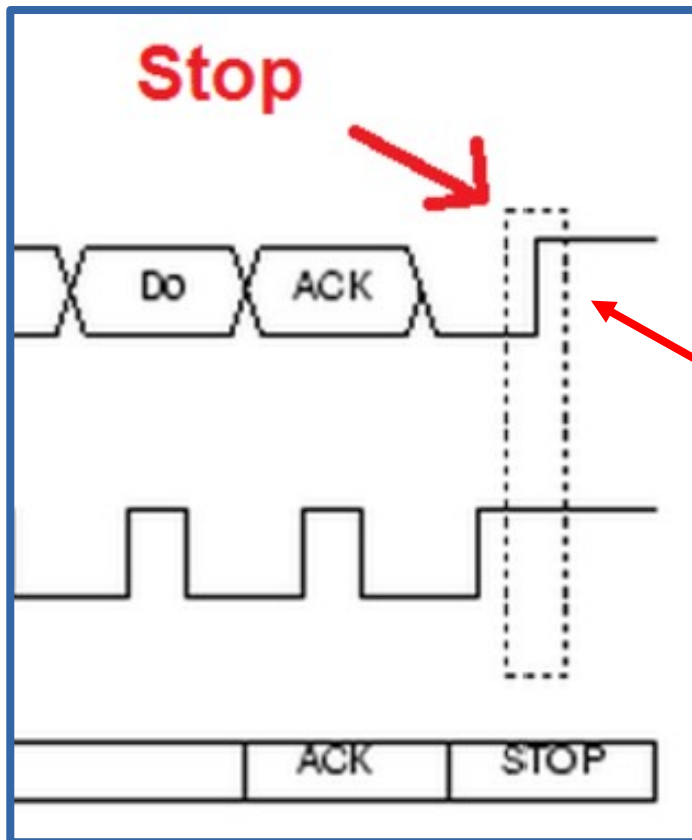
Comunicação serial I2C

- Com a confirmação o Master envia os dados.



Comunicação serial I2C

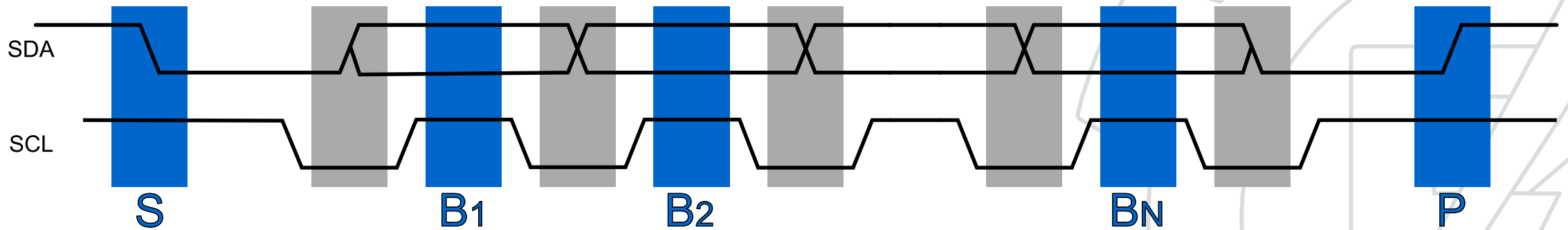
- Depois de receber um novo ACK a transmissão é finalizado com um STOP bit.



SDA sobe com SCL
em nível alto

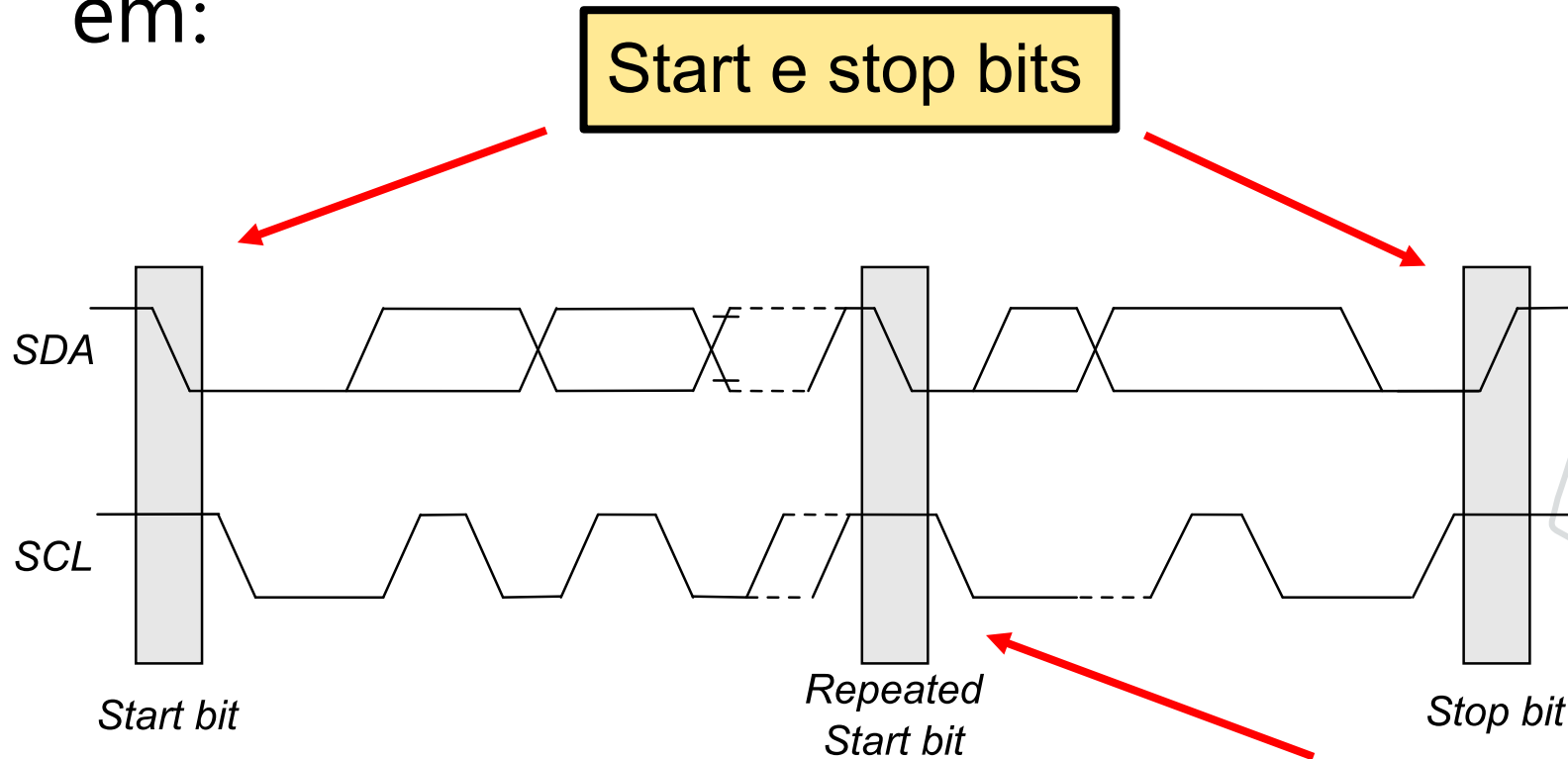
Comunicação serial I2C

- Em azul as informações válidas.
- Em cinza os momentos em que o sinal de dados pode mudar (com clock baixo).



Comunicação serial I2C

O sinal de dados pode mudar com o sinal de clock alto em:



Repeated start
uma condição especial do start bit

Comunicação serial I2C

- Em algumas situações um dispositivo pode estar ocupado e precisar de mais tempo para ler ou escrever um bit.
 - Nesta situação **ele pode segurar o clock em nível baixo** pelo tempo que precisar.
 - Isto é chamado de clock stretching.
- Na transmissão dos dados é preciso verificar se o canal de clock está disponível antes de prosseguir com a leitura ou escrita dos dados.

Comunicando através do I2C

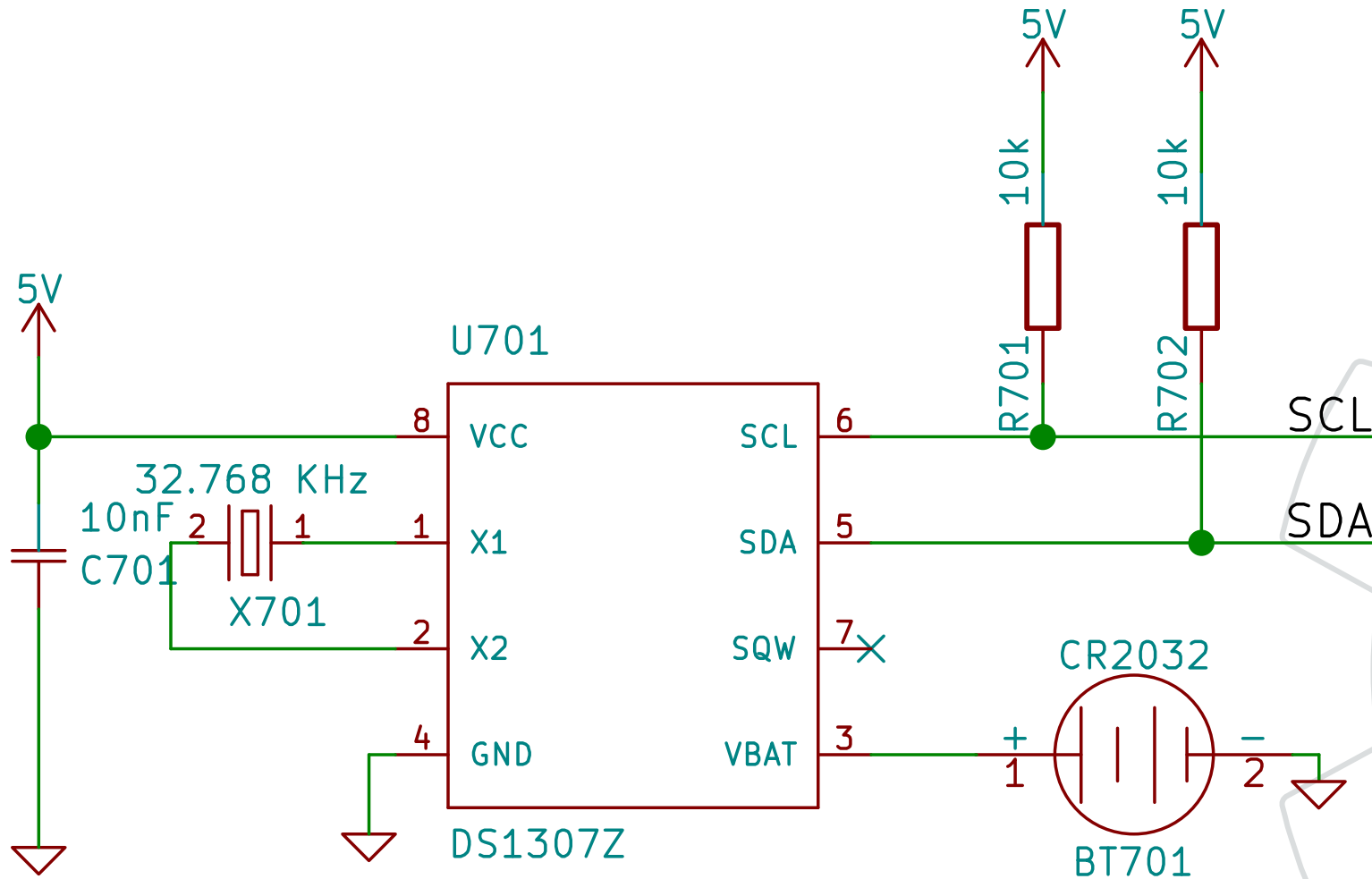


Exemplo de utilização do I2C

- Relógio de tempo real ou RTC (real time clock)
 - É um circuito integrado com um relógio.
 - Geralmente utiliza seu próprio cristal para ficar independente do sistema principal.
 - Pode conter, além do relógio, um calendário.



Ligação dos dispositivos via I2C



Comunicando através do I2C

Informações do datasheet

Slave Receiver Mode (Write Mode):
After each byte is received an acknowledge is recognized as the beginning and end of a reception of the slave address and direct data received after the master generates the slave address. DS1307 address, which is 1101000, follows receiving and decoding the slave address. The DS1307 acknowledges the slave address. DS1307 This sets the register pointer on

Mapa de memória interna

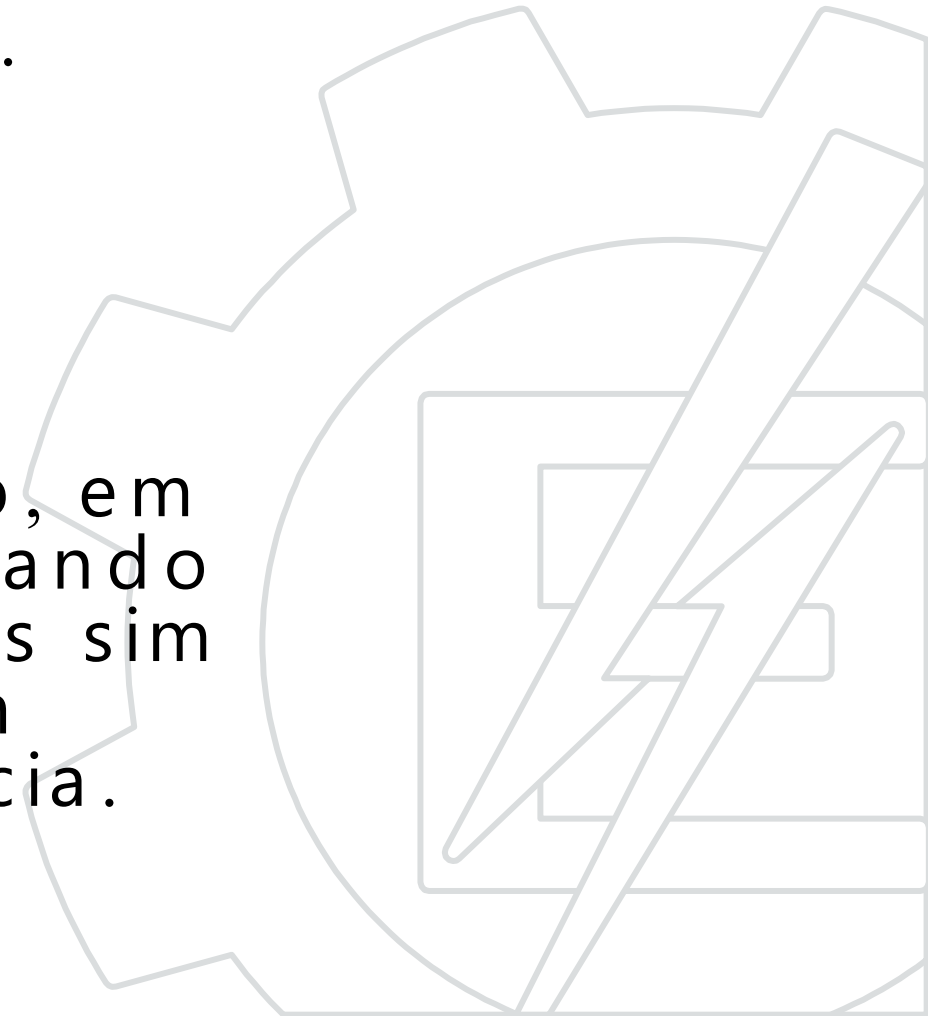
Table 2. Timekeeper Registers

ADDRESS	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	FUNCTION	RANGE
00H	CH	10 Seconds			Seconds				Seconds	00-59
01H	0	10 Minutes			Minutes				Minutes	00-59
02H	0	12	10 Hour	10 Hour	Hours				Hours	1-12 +AM/PM 00-23
		24	PM/AM							
03H	0	0	0	0	0	DAY			Day	01-07
04H	0	0	10 Date		Date				Date	01-31
05H	0	0	0	10 Month	Month				Month	01-12
06H	10 Year				Year				Year	00-99
07H	OUT	0	0	SQWE	0	0	RS1	RS0	Control	—
08H-3FH									RAM 56 x 8	00H-FFH

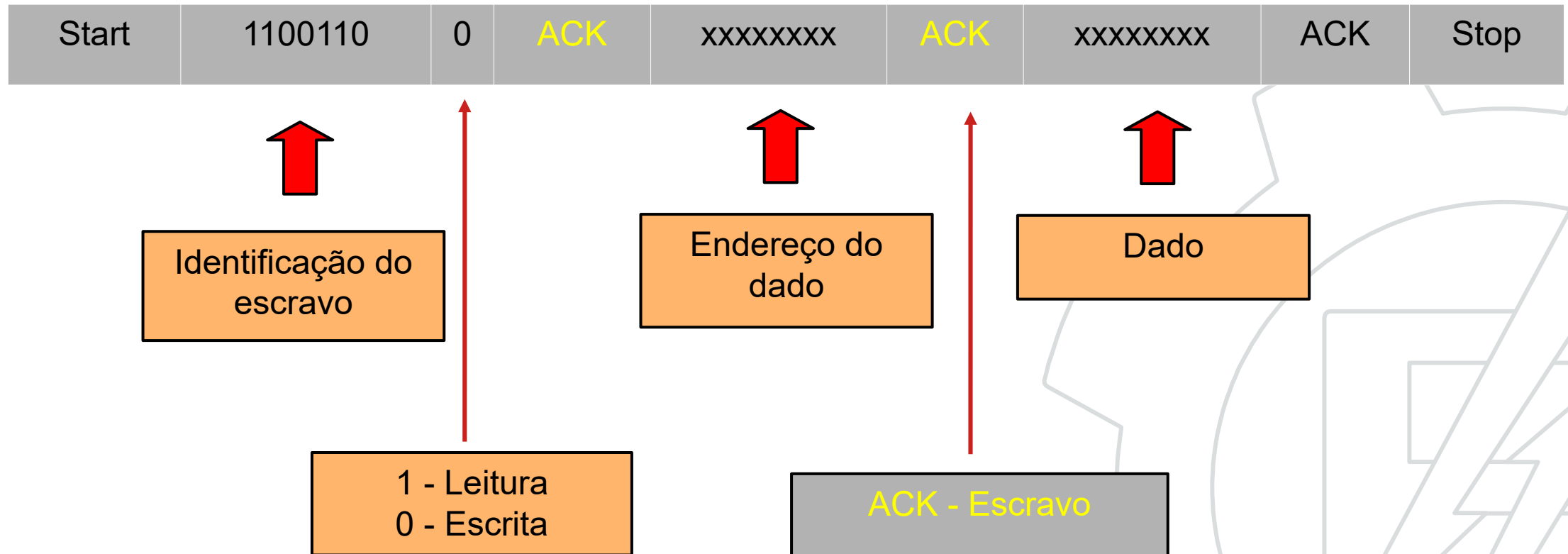
0 = Always reads back as 0.

Comunicando através do I2C

- O procedimento de leitura
 - Envia-se o endereço a ser lido.
 - Realiza-se a leitura do valor.
- Cada procedimento é um comando diferente
 - Para simplificar a comunicação, em alguns casos no primeiro comando não é enviado o “stop bit” mas sim um “repeated start”, dando um segundo comando na sequência.

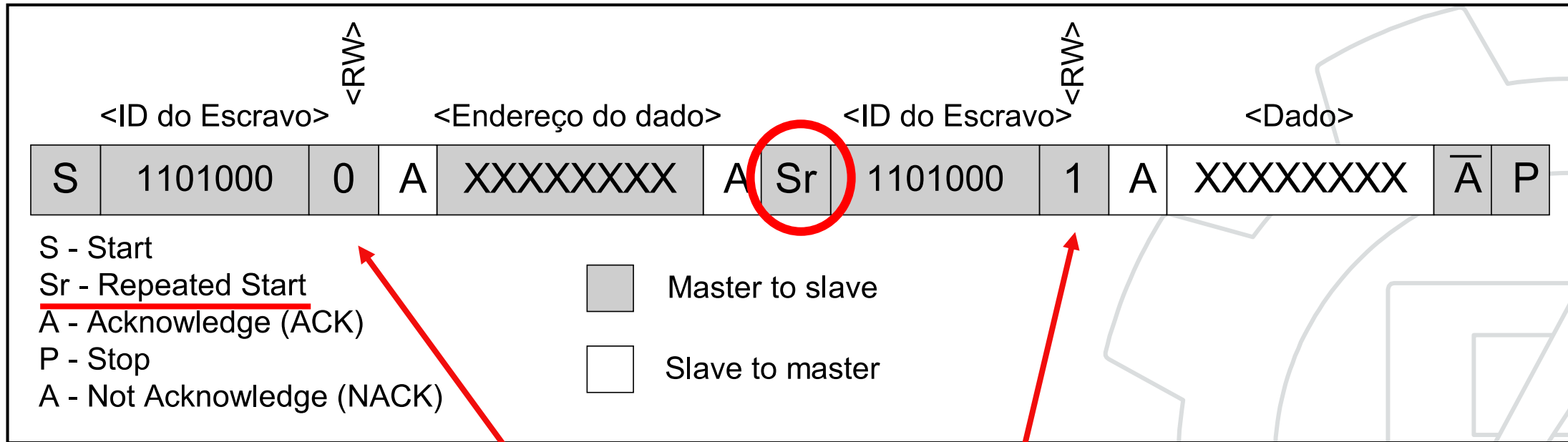


Exemplo de comunicação



Exemplo de comunicação

Leitura de um endereço específico

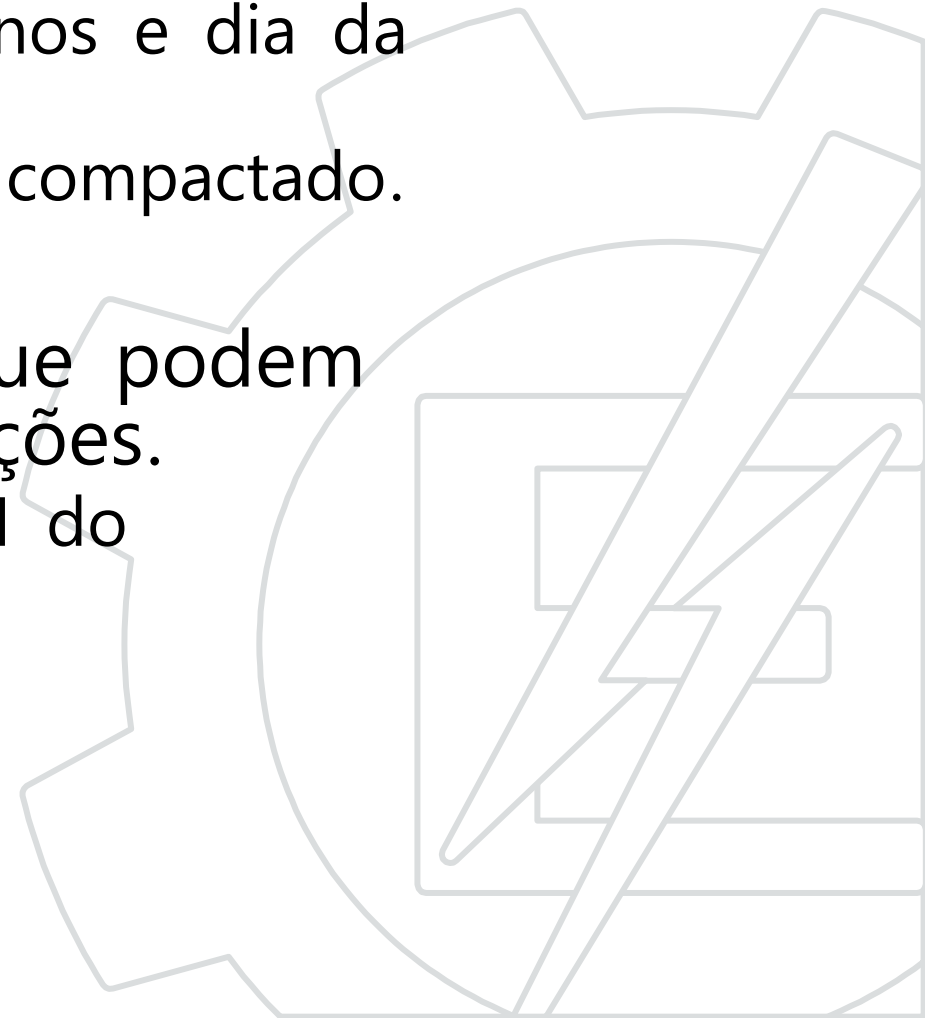


Escrita

Leitura

Uso do DS1307 através do I2C

- No DS1307 7 registros podem ser lidos/escritos
 - Segundo, minutos, horas, dias, meses, anos e dia da semana.
 - Estes valores estão codificados em BCD compactado.
- O DS1307 possui 56 bytes de RAM que podem ser utilizados para armazenar informações.
 - A bateria mantém a informação na RAM do dispositivo.



Comunicando através do I2C

Mapa de memória interna

Endereço	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0	Função	Faixa
00h	CH	Dezena de segundos			Segundos				Seconds	00–59
01h	0	Dezenas de minutos			Minutos				Minutes	00–59
02h	0	12(1) ou 24(0)	10 Horas	10 Horas	Horas				Horas	1–12 +AM/PM 00–23
			PM/ AM							
03h	0	0	0	0	0	Dia/Semana			Dia/Semana	01–07
04h	0	0	Dezenas de Dias		Dias				Dias	01–31
05h	0	0	0	10 Meses	Meses				Meses	01–12
06h	Dezenas de Anos				Anos				Anos	00–99
07h	OUT	0	0	SQWE	0	0	RS1	RS0	Controle	—
08h–3Fh	Byte de dados (RAM)								RAM 56 x 8	00h–FFh

0 = Sempre é lido como zero

CH = Clock Halt (habilita contagem)

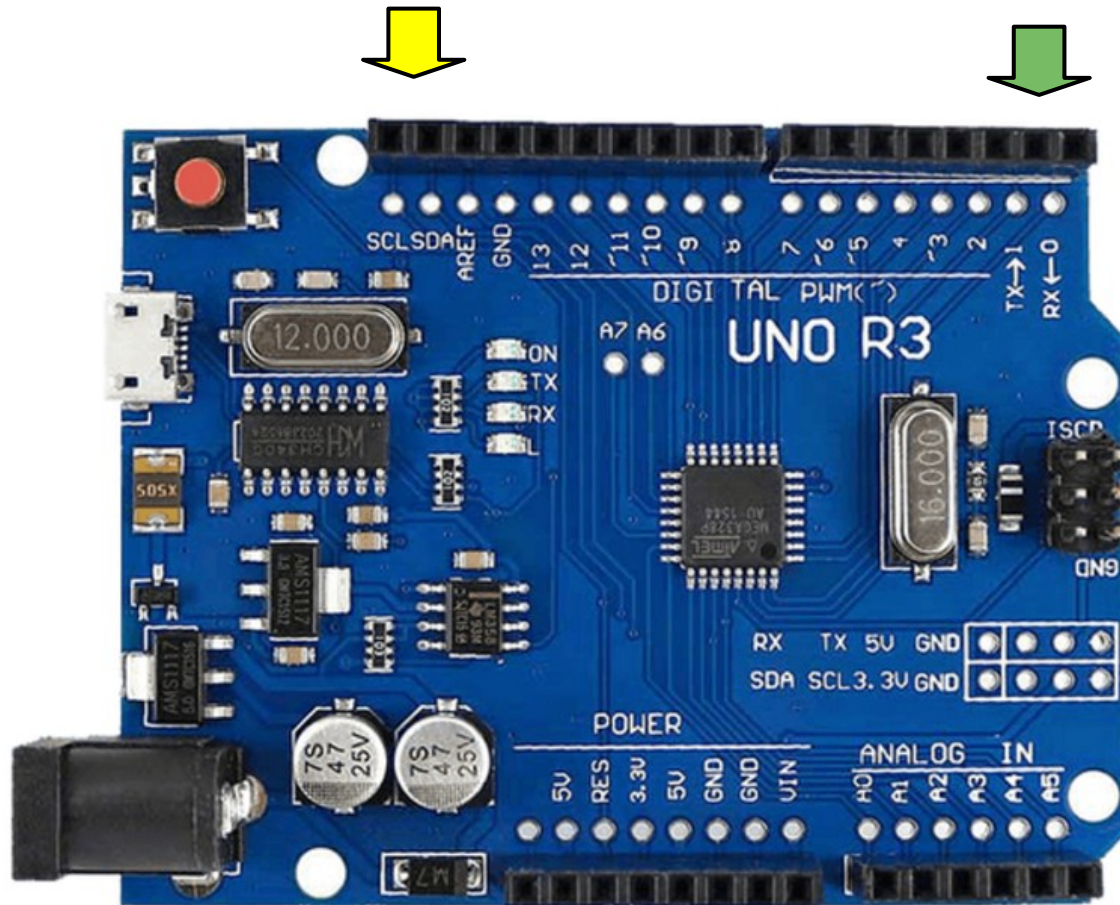
SQWE = Habilita saída do clock

RS = Configura velocidade do clock de saída

Protocollo RS232



Transmissão serial



RS-232

I2C

UART

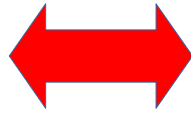
Universal Asynchronous Receiver/Transceiver

	RS-232	RS-485	RS-422
Network Topology	Point-to-Point	Point-to-Point or Multidrop	Point-to-Point or Multidrop
Duplex Type	Full Duplex	Full Duplex Half Duplex	Full Duplex Half Duplex
Number of Devices	1 controller 1 receiver	32 unit loads (controllers or receivers)	1 controller 10 receivers
Signaling	Unbalanced	Balanced (differential signaling)	Balanced (differential signaling)
Max Distance	50 feet at 19.2 Kbps	4000 feet at 100 Kbps	4000 feet at 100 Kbps
Mark (data 1)	-15 to -3 V	1.5 V to 5 V (B > A)	2 V to 6 V (B > A)
Space (data 0)	+3 to +15V	1.5 V to 5 V (A > B)	2 V to 6 V (A > B)

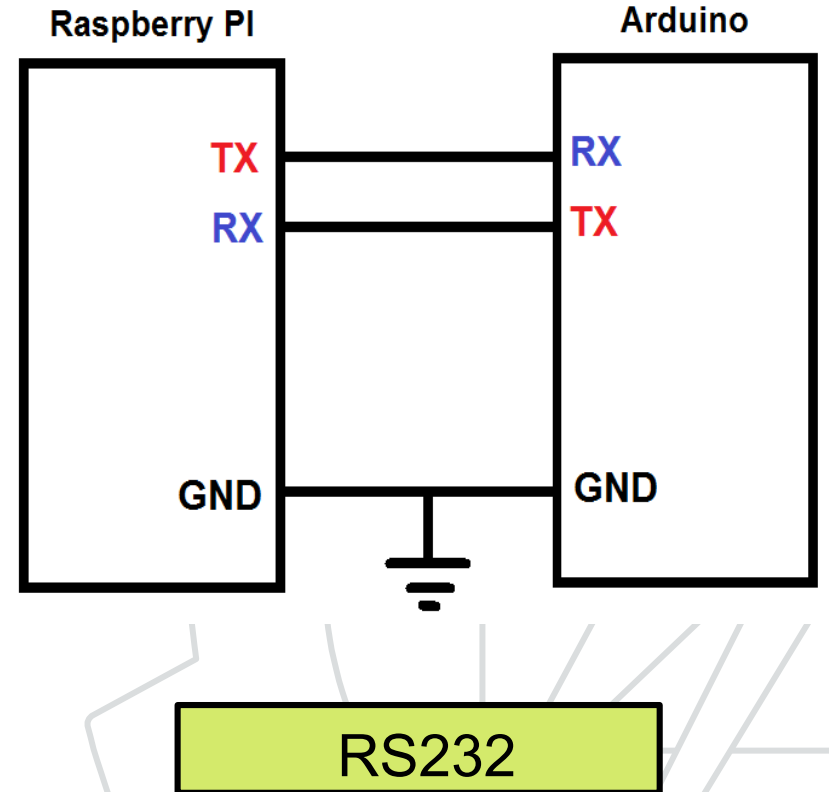
EIA-232 (RS232)



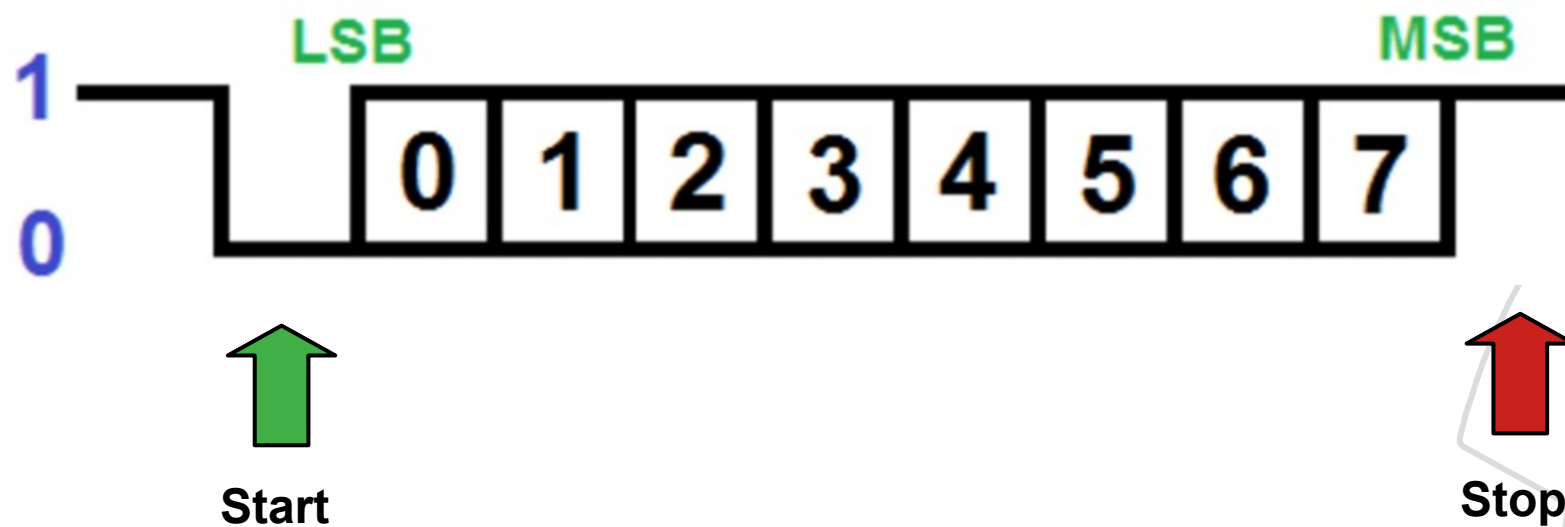
Raspberry Pi



Arduino



Sinal serial RS232

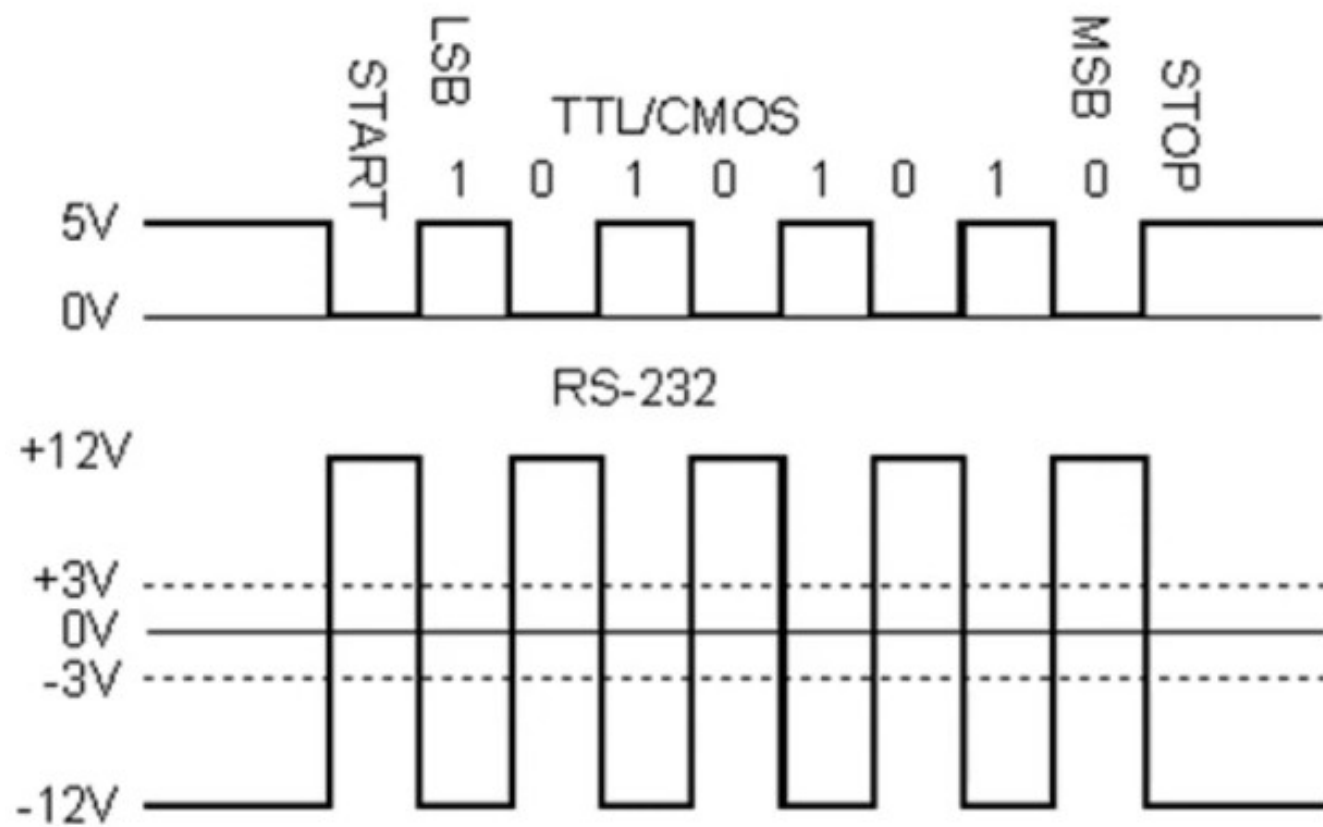


Transmissão assíncrona

5V

0V

Sinal serial RS232

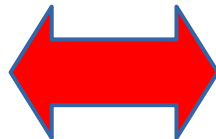


Nível TTL

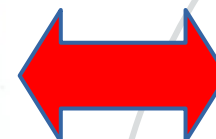
EIA-232

Conversão de nível

TTL
(0V 5V)

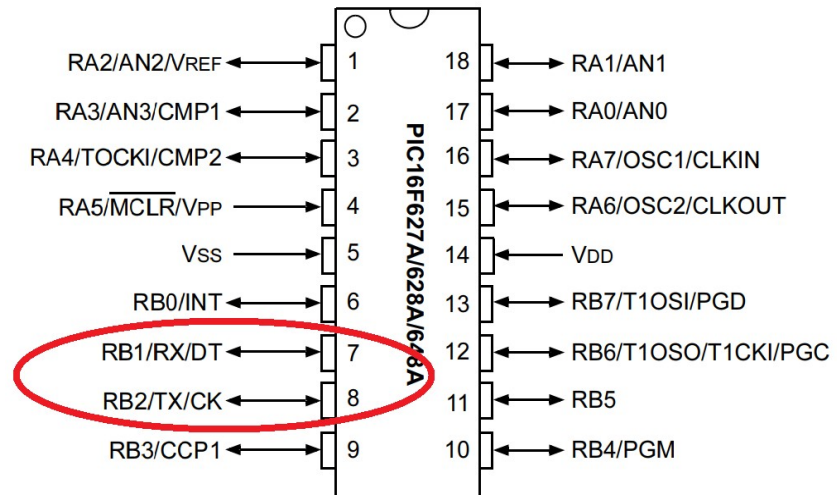


MAX232

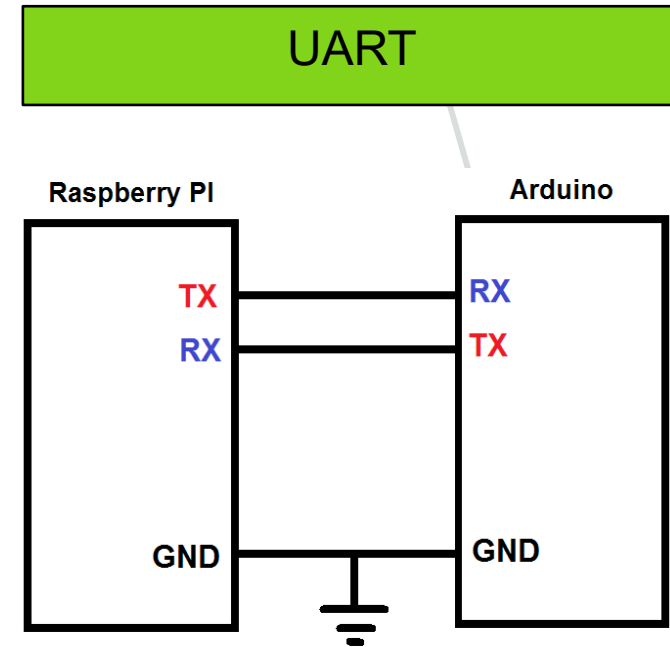


RS232
(12V -12V)

Terminais compartilhados



Entrada e saída digital



Registadores

TABLE 12-2: REGISTERS ASSOCIATED WITH BAUD RATE GENERATOR

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR	Value on all other RESETS
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
18h	RCSTA	SPEN	RX9	SREN	CREN	ADEN	FERR	OERR	RX9D	0000 000x	0000 000x
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

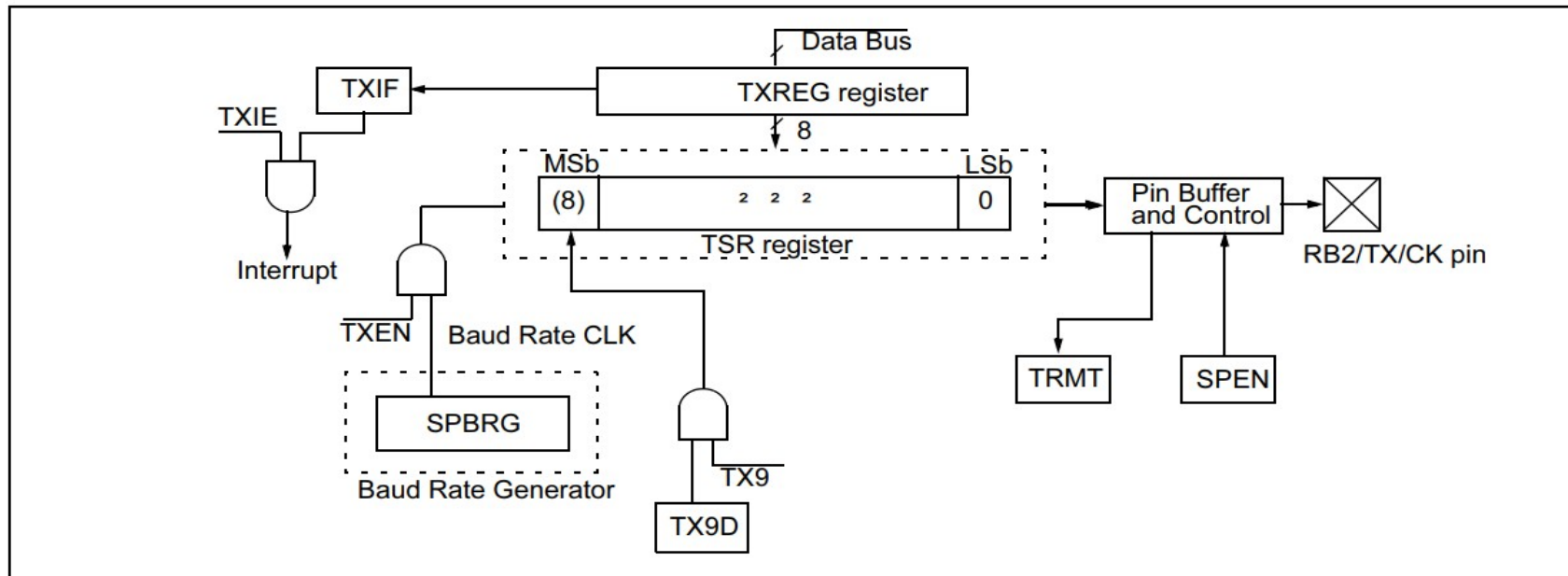
9 bits

Habilita TX

Baud rate

Terminais compartilhados

FIGURE 12-5: USART TRANSMIT BLOCK DIAGRAM



- Ajustar SPBRG com Baud Rate desejado.
- Habilitar TXIE se for utilizar interrupção.
- Se transmissão com 9 bits carregar o bit em TX9D.
- Outros...

O padrão RS232 (base do UART)

Sinal	Significado
SG ou GND	Terra
TD ou TX	Transmissão de dados
RD ou RX	Recepção de dados
DTR	Terminal de dados pronto
DSR	Conjunto de dados pronto
RTS	Pronto para enviar(computador)
CTS	Envie os dados (modem)
DCD	Portadora detectada
RI	Indicador de telefone tocando
FG	(Frame Ground)

- **Modos de operação**
- Síncrono ou assíncrono.
- Pinos auxiliares RTS e DTR.
- Bit de paridade.
- Taxa de transmissão.
- Stop bit.

Mais usado

- **8N1**
8 - 8 bits de dados
N - Sem paridade
1 - Um stop bit

Exemplo de programa (PIC16F626A)

```
#define BAUD 9600
#define FOSC 4000000L
#define DIVIDER ((int)(FOSC/(16UL * BAUD) -1))
```

```
void InitUART()
```

```
{
    TRISB1 = 1; // Pino RX
    TRISB2 = 0; // Pino TX
    SPEN = 1; // Desabilita os pinos
    SPBRG = DIVIDER; // Ajusta o Baud Rate
    BRGH = 1;
    SYNC = 0; // Modo assíncrono
    CREN = 1; // Habilita recepção contínua
    TXIE = 0; // Desabilita interrupção
    RCIE = 1; // Habilita interrupção para recepção
    TX9 = 0; // Modo transmissão em 8 bits
    RX9 = 0; // Modo recepção em 8 bits
    TXEN = 0; // Reseta Transmissor
    TXEN = 1; // transmit enabled
    SPEN = 1; // Habilita os pinos
}
```

Calculando valor

Ajustando SPBRG

Modo 8 bits

Ajustando os parâmetros da porta

Exemplo de transmissão

```
void envia_byte(unsigned char dado)
{
    while(!TRMT);
    TXREG = dado;
}
```

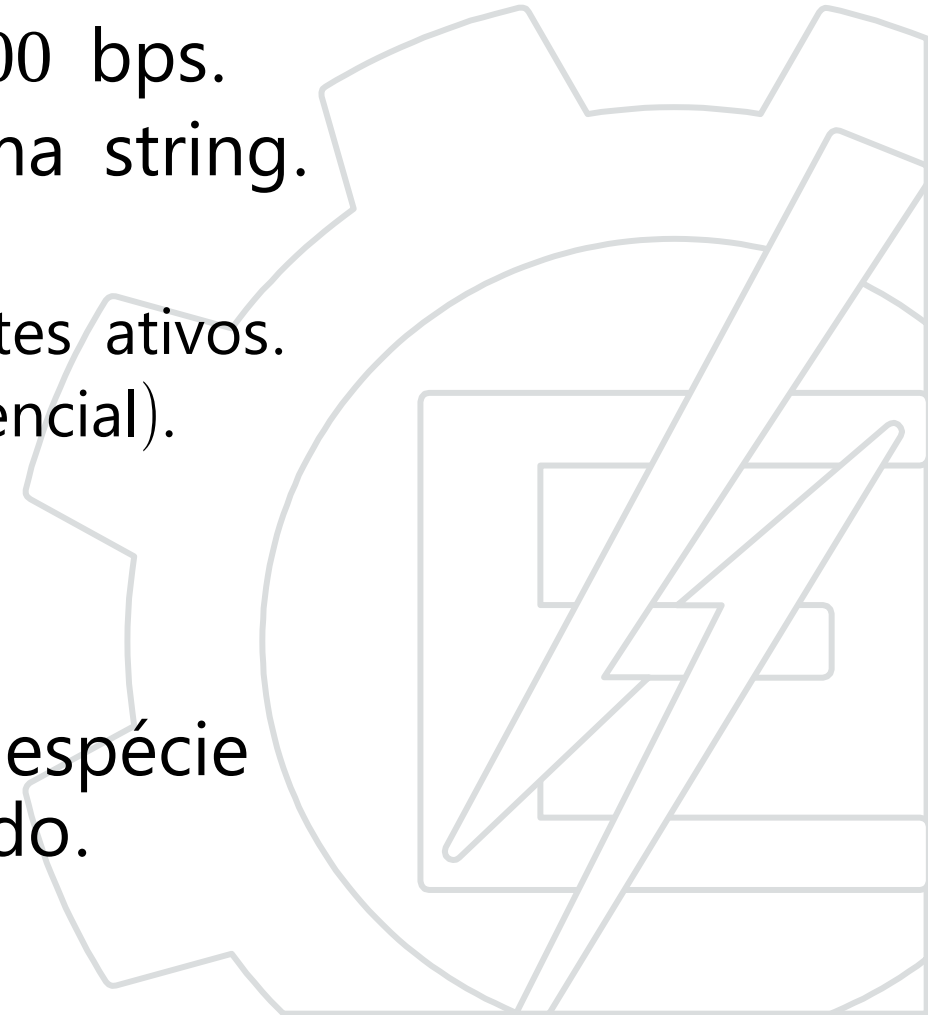
Espera o dado anterior
ser transmitido

Comunicando através da UART/RS232

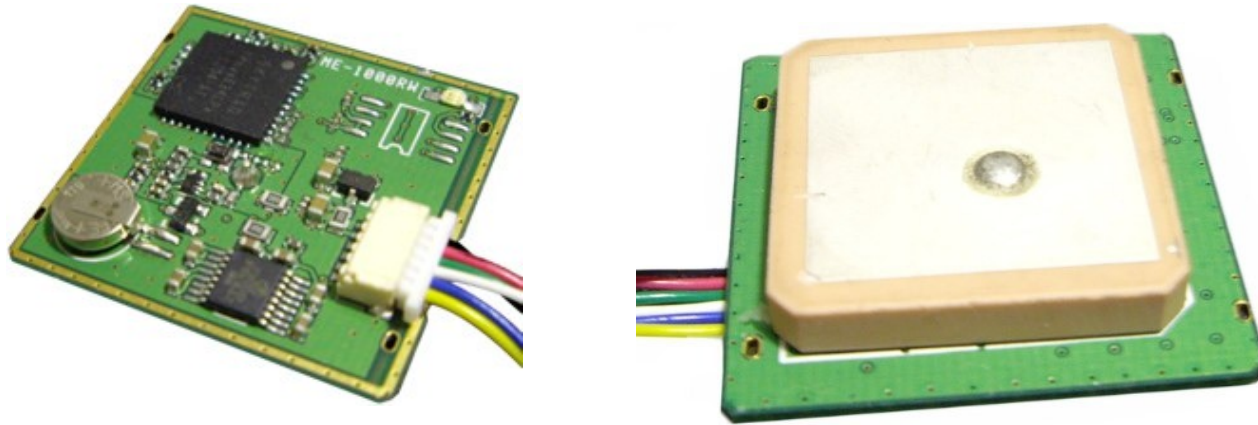


Exemplo de interface com dispositivo

- GPS integrado com antena ME-1000RW.
 - Comunica-se usando padrão UART 9600 bps.
 - Os dados são passados através de uma string.
 - Implementa vários padrões NEMA.
 - GPGSA - Precisão e quantidade de satélites ativos.
 - GPGCA - Dados fixados (lat/long/alt/diferencial).
 - GPRMC - Dados mínimos (lat/long/alt).
 - GPVTG - Velocidade e deslocamento.
 - GPGSV - Dados dos satélites visíveis.
 - Cada um destes padrões possui uma espécie de protocolo que deve ser decodificado.



Exemplo de interface com dispositivo



GPS integrado com antena ME-1000RW

- Comunicação UART 9600 bps
 - por strings
- GPGSA - Satélites ativos.
- GPGCA - Lat/long/alt/diferencial.
- GPRMC - lat/long/alt.
- GPVTG - Velocidade e deslocamento.
- GPGSV - Satélites visíveis.

Mensagem do GPRMC

- Formada por 14 campos:
 - 0 - ID da mensagem.
 - 1 - Tempo (UTC).
 - 2 - Rastreando (R) ou Aceitável (A).
 - 3 - Latitude.
 - 4 - Norte/Sul.
 - 5 - Longitude.
 - 6 - Leste/Oeste (E/W).
 - 7 - Velocidade (magnitude).
 - 8 - Velocidade (ângulo).
 - 9 - Data (UTC).
 - 10 - Variação Magnética (ângulo em graus).
 - 11 - Variação Magnética (direção, E/W).
 - 12 - Modo de operação (N não válido, A autônomo, D diferencial, E estimado, M manual, S simulação).
 - 13 - Checksum.



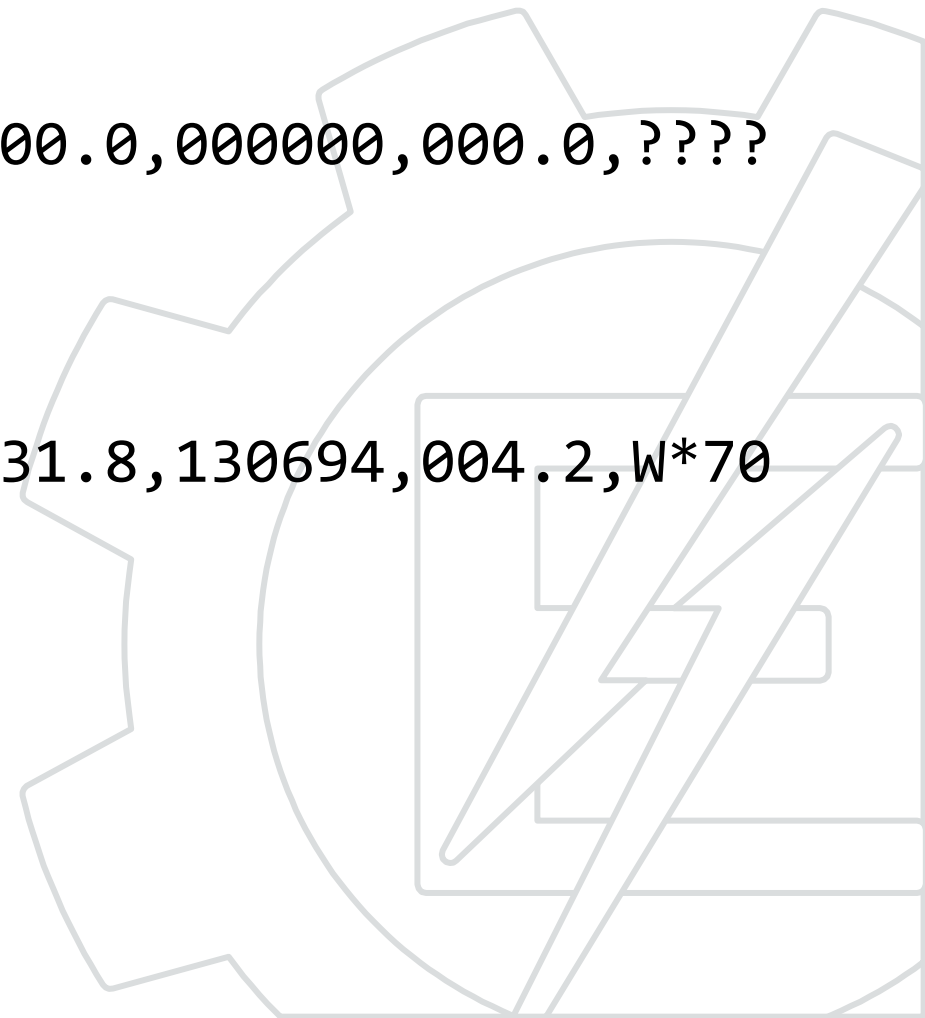
Mensagem do GPRMC

- Formato:

\$IDENT,000000,X,0000.00,X,00000.00,X,000.0,000.0,000000,000.0,????

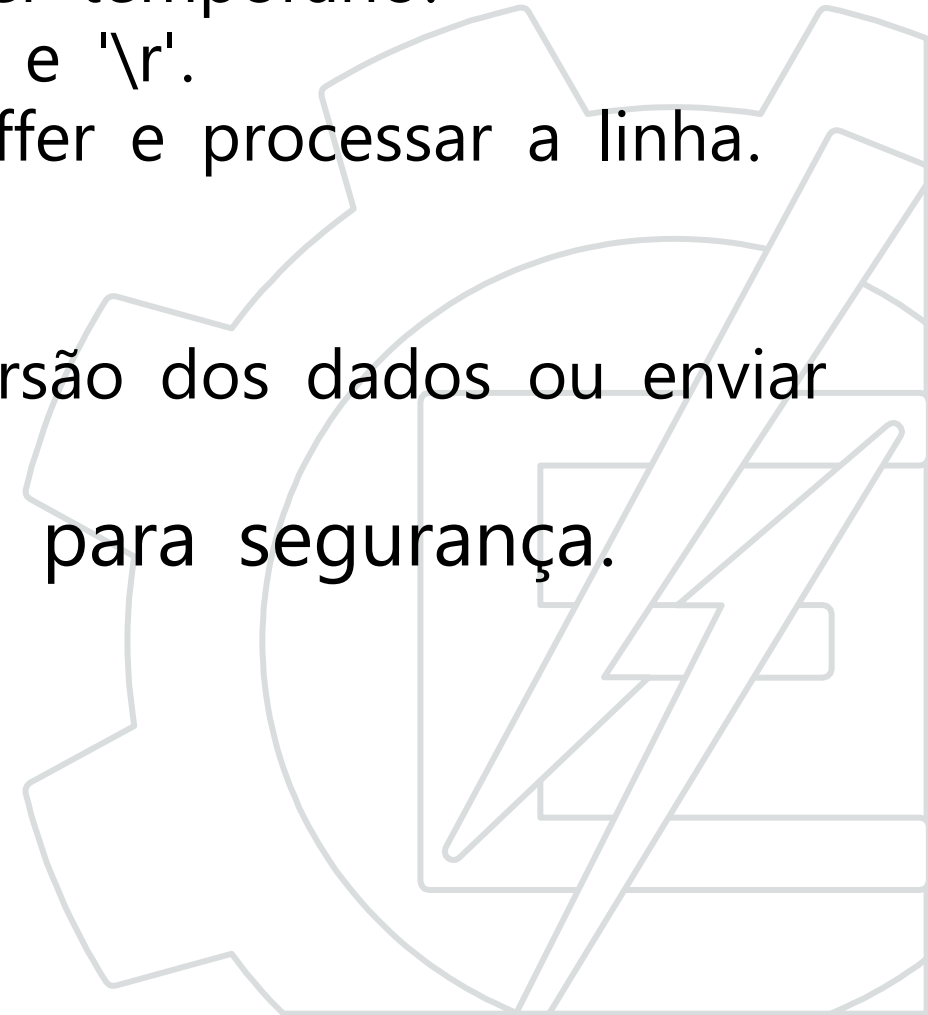
- Exemplo

\$GPRMC,220516,A,5133.82,N,00042.24,W,173.8,231.8,130694,004.2,W*70

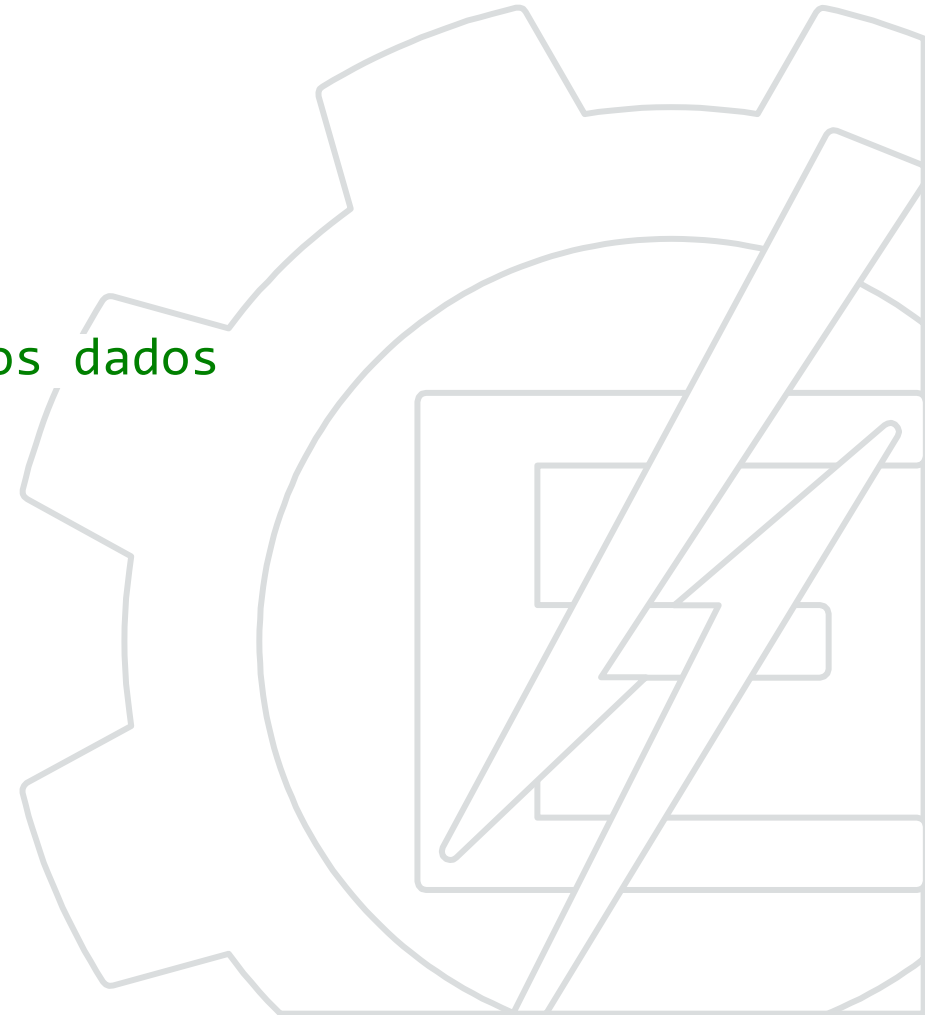


Etapas do programa

- Rotina para recepção dos dados
 - Armazenar cada byte recebido num buffer temporário.
 - Aguardar dois bytes de fim de linha '\n' e '\r'.
 - Caso chegar "fim de linha", resetar o buffer e processar a linha.
 - Verificar o ID da mensagem (\\$GPRMC).
 - Cada campo é separado por vírgulas.
 - Percorrer o campo atual e realizar conversão dos dados ou enviar p/ o LCD.
- Não esquecer de verificar o checksum para segurança.



```
#include "serial.h"
#include "lcd.h"
void main(void) {
    unsigned char pos=0; //posição atual do buffer
    char buffer[100]; //buffer temporário
    char data;
    serialInit();
    SPBRG = 207; //configura para 56k
    lcdInit();
    for(;;) {
        data = serialRead(); //recebimento dos dados
        if (data != 0) {
            buffer[pos] = data;
            pos++;
            if (pos >= 100) { pos = 0; }
        }
        // Processamento do buffer
    }
}
```



```
//teste de fim de mensagem
if((pos > 2) &&
    (buffer[pos-2] == 13) && //CR
    (buffer[pos-1] == 10)) { //LF
    pos = 0;
    //se o ID estiver correto processa o buffer
    if( (buffer[0] == '$') &&
        (buffer[1] == 'G') &&
        (buffer[2] == 'P') &&
        (buffer[3] == 'R') &&
        (buffer[4] == 'M') &&
        (buffer[5] == 'C'))
    {
        // parser da mensagem $GPRMC
        // ...
    }
}
```



```
//começa depois do campo ID
pos = 7;

//1 - Hora UTC
while(buffer[pos] != ','){
    //EnviaDados(buffer[pos]);
    pos++;
}
pos++; //pula a vírgula

//2 A(OK)
while(buffer[pos] != ','){
    //EnviaDados(buffer[pos]);
    pos++;
}
pos++; //pula a vírgula
```



```
//3-Latitude
```

```
lcdCommand(0x80);
```

```
lcdChar('L');
```

```
lcdChar('A');
```

```
lcdChar('T');
```

```
while(buffer[pos] != ','){  
    lcdChar(buffer[pos]);  
    pos++;  
}
```

```
pos++;//pula a vírgula
```

```
//4 N/S
```

```
while(buffer[pos] != ','){  
    lcdChar(buffer[pos]);  
    pos++;  
}
```

```
pos++;//pula a vírgula
```



//5-Longitude

```
lcdCommand(0xC0);
```

```
lcdChar('L');
```

```
lcdChar('O');
```

```
lcdChar('N');
```

```
while(buffer[pos] != ','){  
    lcdChar(buffer[pos]);  
    pos++;  
}
```

```
pos++; //pula a vírgula
```

```
//6-Leste(E)/Oeste(W)
```

```
while(buffer[pos] != ','){  
    lcdChar(buffer[pos]);  
    pos++;  
}
```

```
pos++; //pula a vírgula
```

```
//..continua para os demais campos
```



Resultado



Até a próxima

